

Reti Neurali

Capire le fondamenta dell'apprendimento automatico non lineare



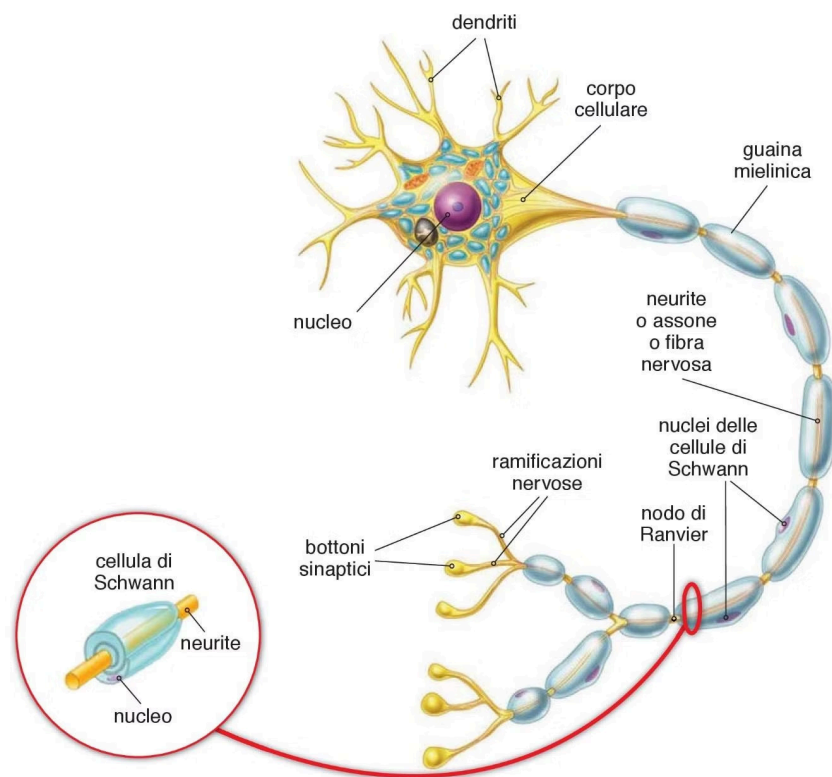
by Daniele Mantovani



Il Neurone Biologico

Il **neurone biologico** è l'unità fondamentale del sistema nervoso, responsabile della trasmissione degli impulsi elettrici. È composto da un corpo cellulare chiamato soma, con dendriti che ricevono i segnali e un assone che li trasmette ad altri neuroni.

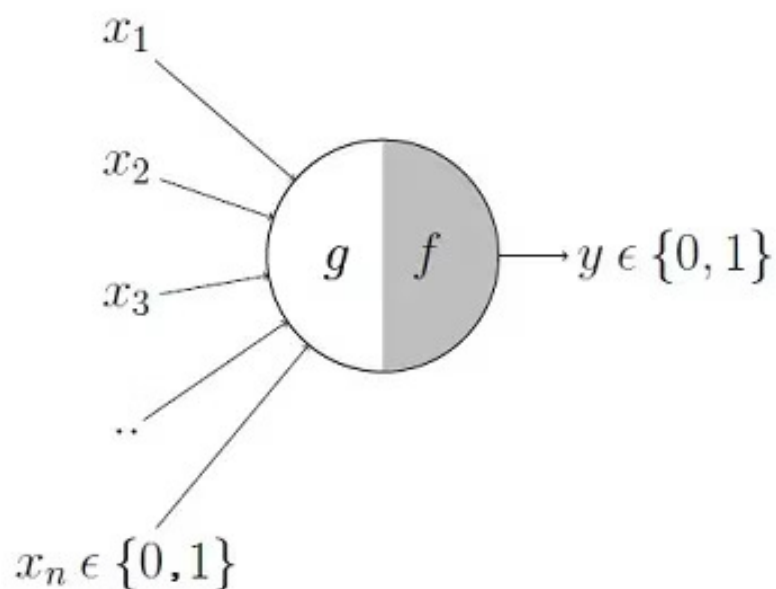
Attraverso la sinapsi, i neuroni comunicano tra loro permettendo l'elaborazione e il trasferimento di informazioni nel nostro corpo. Questa struttura altamente specializzata è alla base del funzionamento del cervello e dell'intero sistema nervoso.



- **Dendriti:** prolungamenti ramificati che ricevono segnali da altri neuroni.
- **Corpo cellulare (soma):** la parte centrale del neurone contenente il nucleo.
- **Nucleo:** struttura all'interno del corpo cellulare che contiene il materiale genetico.
- **Neurite o assone o fibra nervosa:** il lungo prolungamento che trasmette impulsi elettrici dal corpo cellulare verso altre cellule.
- **Guaina mielinica:** uno strato isolante formato da cellule di Schwann, che aumenta la velocità di trasmissione degli impulsi nervosi.
- **Nuclei delle cellule di Schwann:** i nuclei delle cellule che producono la guaina mielinica.
- **Nodo di Ranvier:** piccole interruzioni della guaina mielinica che facilitano la trasmissione saltatoria dell'impulso nervoso.
- **Ramificazioni nervose:** le estremità dell'assone che trasmettono segnali ai neuroni successivi.
- **Bottoni sinaptici:** strutture terminali che rilasciano neurotrasmettitori per comunicare con altri neuroni o cellule bersaglio.
- **Cellula di Schwann:** cellula gliale che avvolge l'assone formando la guaina mielinica.

Neurone di McCulloch e Pitts

Il [neurone artificiale di McCulloch e Pitts](#) è considerato il progenitore delle moderne reti neurali. Sviluppato negli anni '40, questo modello matematico semplificato del neurone biologico fu un importante passo verso la comprensione del funzionamento del cervello umano.



$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

- θ = soglia di attivazione
- y = output attivo/spento (1/0)
- x_i = ingresso attivo/spento (1/0)

E' in grado di svolgere alcune semplici operazioni logiche, come ad esempio

AND

Il neurone si attiva solo se *tutti* gli input sono attivi. Se consideriamo ad esempio due input x_1, x_2 , è sufficiente porre $\theta = 2$.

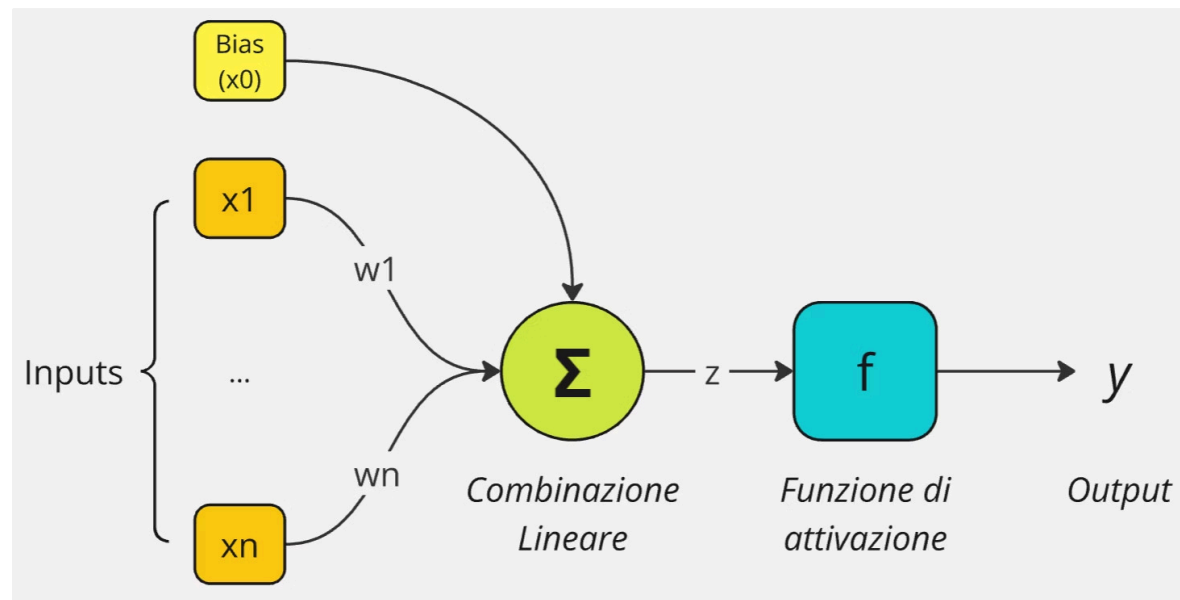
OR

Il neurone si attiva se *almeno uno* degli input è attivo. Se consideriamo ad esempio due input x_1, x_2 , è sufficiente porre $\theta = 1$.

Percettrone di Rosenblatt

Nel 1957, [Frank Rosenblatt](#), uno psicologo statunitense, descrive il modello del [percettrone](#) in un report intitolato "[The Perceptron: A Perceiving and Recognizing Automaton](#)". L'innovazione principale rispetto al modello di McCulloch-Pitts riguarda la presenza di **input non solo binari e associati a pesi numerici**.

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$



Dove:

- x_i rappresenta gli input, con i che varia da 1 a n (il numero di feature).
- w_i rappresenta i pesi associati a ciascun input.
- b è il bias.
- La somma ponderata viene passata a una funzione di attivazione f a gradino (θ è la soglia di attivazione):

$$y = f(x) = \begin{cases} 1 & z \geq \theta \\ 0 & z < \theta \end{cases}$$

Algoritmo di apprendimento

L'apprendimento del Perceptron si basa sulla **regola di aggiornamento dei pesi**, detta anche **regola di apprendimento di Hebb modificata**:

- **Inizializzazione:** I pesi w_i sono inizializzati a valori casuali piccoli o a zero.
- **Calcolo dell'output:** Per ogni esempio di addestramento, si calcola il valore di y usando la funzione di attivazione.
- **Calcolo dell'errore:**

$$e = d - y$$

dove d è il valore desiderato (output corretto) e y è l'output calcolato.

- **Aggiornamento dei pesi:**

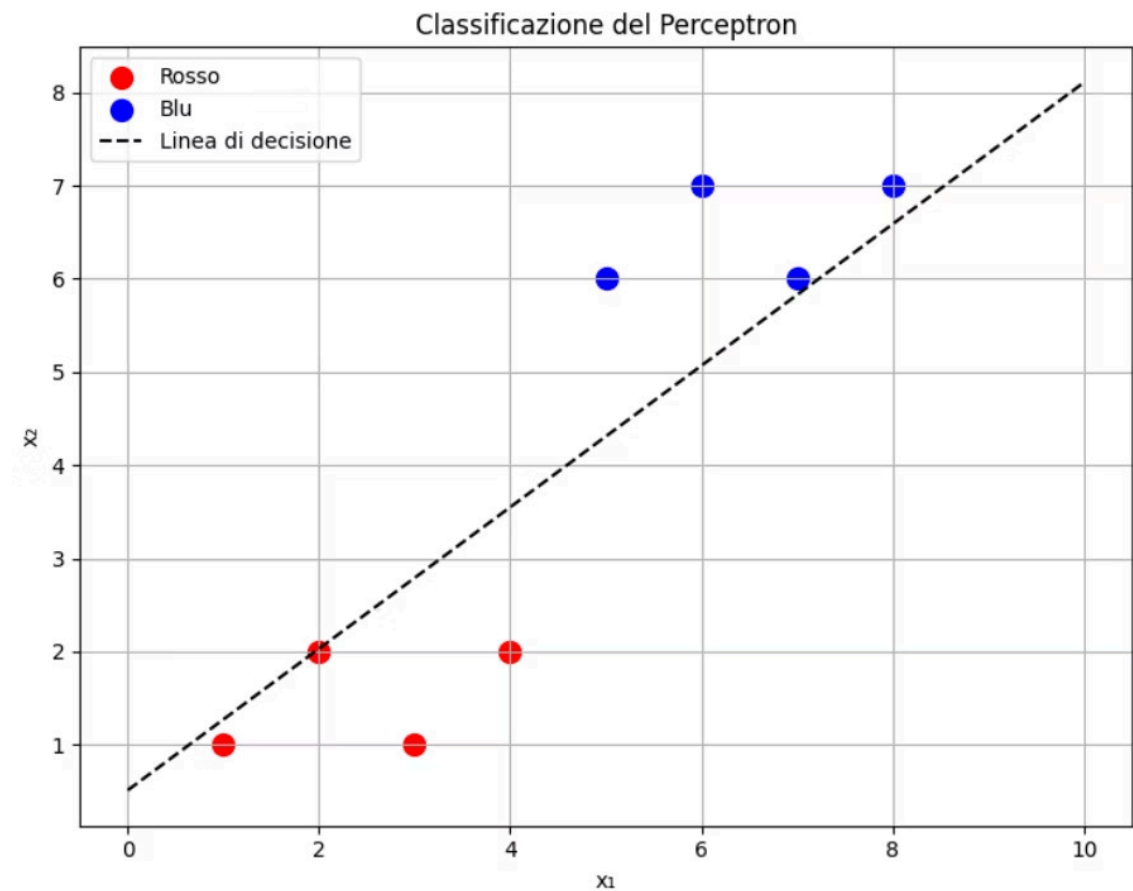
$$w_i^{(t+1)} = w_i^{(t)} + \eta \cdot e \cdot x_i$$

dove:

- η è il **learning rate** (solitamente un valore piccolo, es. 0.01),
- e è l'errore tra output previsto e desiderato,
- x_i è l'input corrispondente.
- **Ripetizione:** Il processo si ripete su tutti i dati di addestramento fino a quando l'errore non si annulla (o diventa accettabilmente piccolo).

Cosa può fare un Percettrone

x_1	x_2	Colore
1	1	Rosso
2	2	Rosso
3	1	Rosso
4	2	Rosso
5	6	Blu
6	7	Blu
7	6	Blu
8	7	Blu



$$w_1 = -0.28608925 \quad (1)$$

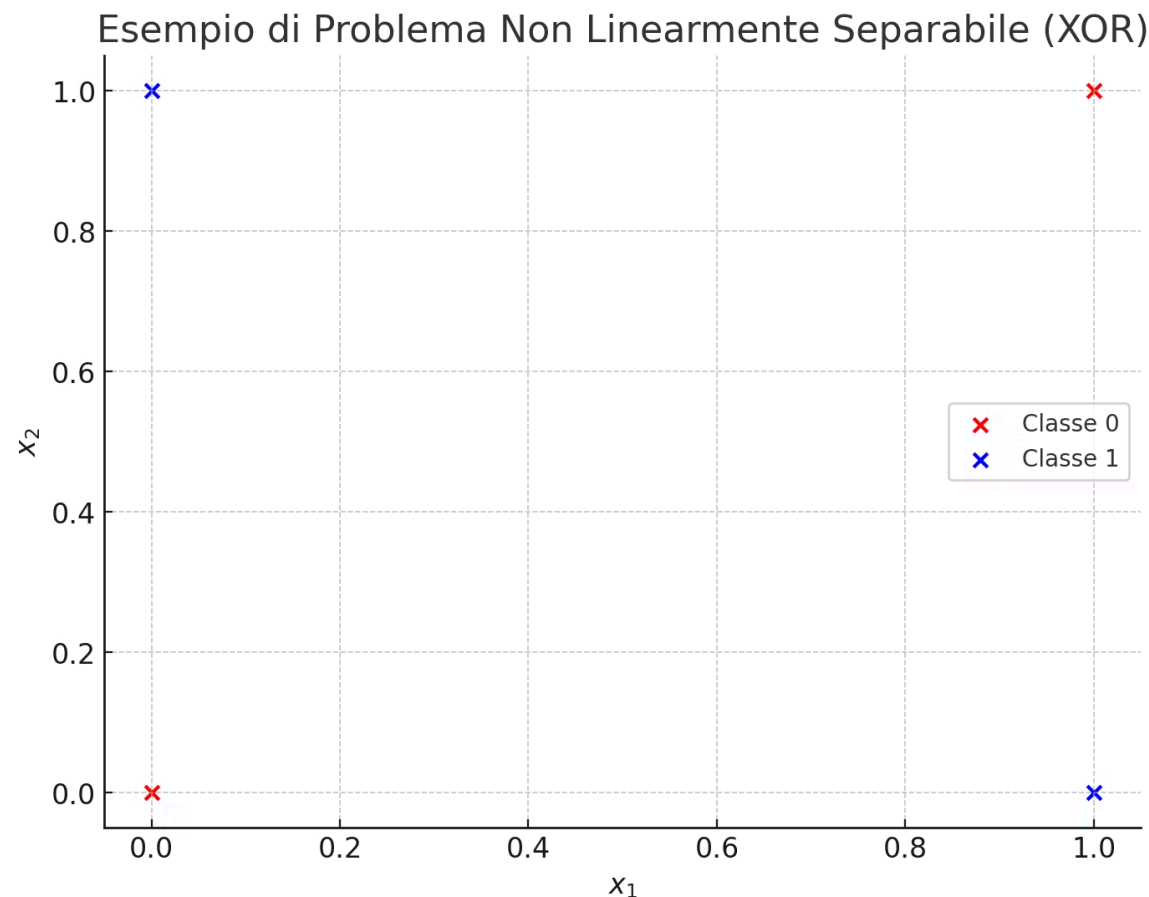
$$w_2 = 0.3765268 \quad (2)$$

$$b = 0.19188430659776917 \quad (3)$$

$$f(x) = \begin{cases} Blu & z \geq 0 \\ Rosso & z < 0 \end{cases}$$

Cosa non è in grado di fare un Percettrone

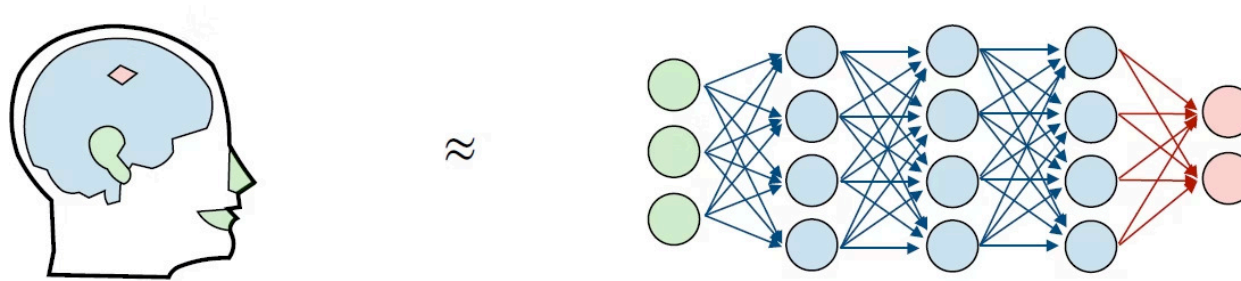
[Marvin Minsky](#), insieme a [Seymour Papert](#), ha svolto un ruolo cruciale nello sviluppo delle reti neurali artificiali. Nel 1969, pubblicarono il libro "[Perceptrons](#)", che analizzava matematicamente le limitazioni dei perceptroni. Questo lavoro dimostrò che i perceptroni erano in grado di eseguire solo operazioni elementari, ad esempio non era in grado di separare i punti che rappresentano gli output della funzione logica XOR.



- Questo limite ha portato allo sviluppo di modelli di reti neurali più complessi (con più strati, chiamati **reti neurali multistrato** o **Multilayer Perceptron**) che possono, ad esempio, risolvere problemi di classificazione più complicati.

Cos'è una Rete Neurale?

Una rete neurale è un modello computazionale ispirato al funzionamento del cervello umano. È composta da unità chiamate "neuroni" che sono interconnessi e si scambiano segnali per elaborare *input* e produrre *output*. I neuroni sono organizzati in strati, tra cui uno di *input*, uno di *output* e, a volte, strati intermedi.



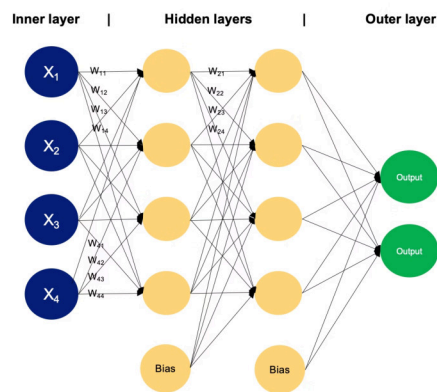
Le reti neurali trovano applicazione in numerosi campi, come il **riconoscimento immagini**, la **traduzione automatica** e la **diagnostica medica**, grazie alla loro capacità di apprendere e riconoscere modelli complessi dai dati.

Architettura della Rete

Le reti neurali possono avere diverse architetture. Alcuni esempi:

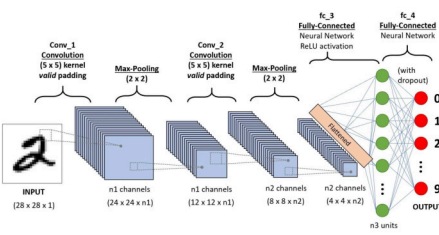
Multi-Layer Perceptron (MLP)

Utilizzata per problemi di classificazione e regressione, come previsione di valori numerici o classificazione di dati tabellari.



Reti Neurali Convoluzionali (CNN)

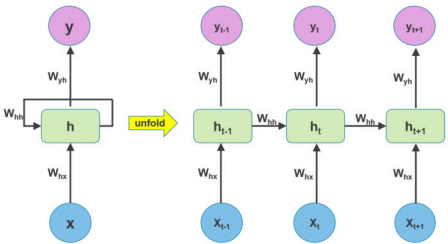
Popolarmente impiegate per il riconoscimento di immagini e video, perché riescono a rilevare pattern visivi, come bordi, forme e texture.



[CNN Explainer](#)

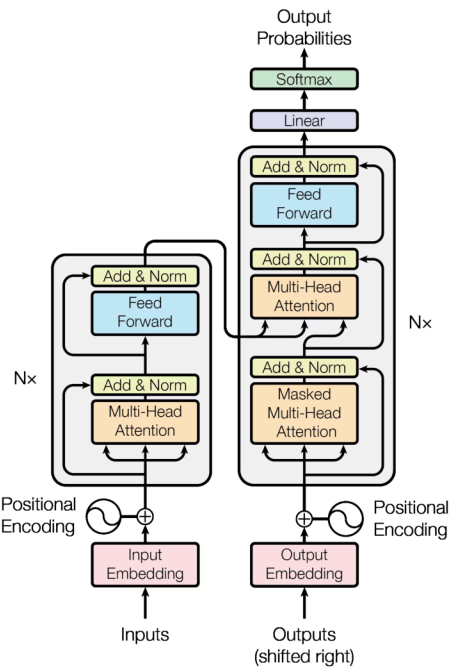
Reti Neurali Ricorrenti (RNN)

Utilizzate per dati sequenziali, come il riconoscimento vocale e la traduzione automatica.



Transformer

Utilizzata per problemi di linguaggio naturale, come traduzione automatica, riassunti di testi, chatbot.



Apprendimento Supervisionato

- Definiamo il **modello** come una funzione:

$$\mathbf{y} = \mathbf{f}[\mathbf{x}, \phi]$$

Dove:

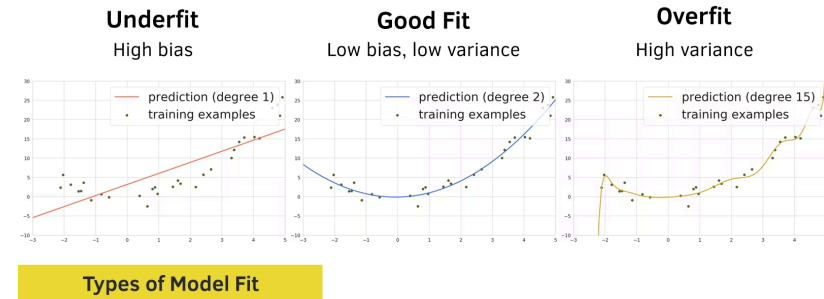
- x è l'input
 - y è la previsione
 - ϕ è l'insieme dei parametri del modello.
- Il processo di calcolo della previsione y a partire dall'input x è detto **inferenza**.
- Addestramento**: l'obiettivo dell'addestramento (o **apprendimento**) del modello f è trovare un insieme di valori per i parametri ϕ che consentano una previsione sensata y a partire da un certo input x .
- I parametri vengono appresi a partire da un certo insieme di esempi, detto **training set**, composto da coppie $I = \{\mathbf{x}_i, \mathbf{y}_i\}$. L'obiettivo è selezionare parametri che associno ogni input di training al suo output associato il più fedelmente possibile.
- Definiamo **loss** L (**perdita**) come un valore scalare che riassume quanto male il modello predice gli output di training dai loro input corrispondenti per i parametri ϕ . Possiamo considerare loss come una funzione $L[\phi]$ che dipende dai parametri ϕ (per la precisione dipende anche dal training set).
- Quando si addestra il modello si cercano parametri $\hat{\phi}$ che minimizzano la loss function:

$$\hat{\phi} = \arg \min_{\phi} [L[\phi]]$$

- Se, dopo la fase di addestramento, il valore di loss è sufficientemente piccolo, allora il modello calcola predizioni accurate per gli esempi del training set.

Valutazione e Generalizzazione

- Dopo aver addestrato un modello, dobbiamo ora valutarne le prestazioni; eseguiamo il modello su dati di test (**test set**) differenti da quelli del training set per vedere quanto bene **generalizza** da esempi che non ha osservato durante l'addestramento. Se le prestazioni sono adeguate, allora siamo pronti a distribuire il modello.
- Un modello troppo semplice (come una linea retta) potrebbe non essere in grado di catturare la vera relazione tra input e output. Questo fenomeno è noto come **underfitting**.
- Al contrario, un modello molto espressivo può descrivere peculiarità statistiche dei dati di training che sono atipiche e portano a previsioni insolite. Questo fenomeno è noto come **overfitting**.



[Hacker's Guide to Fixing Underfitting and Overfitting Models](#)

1D Linear Regression Model

$$y = f[x, \phi] = \phi_0 + \phi_1 x$$

(l'equazione descrive una famiglia di funzioni)

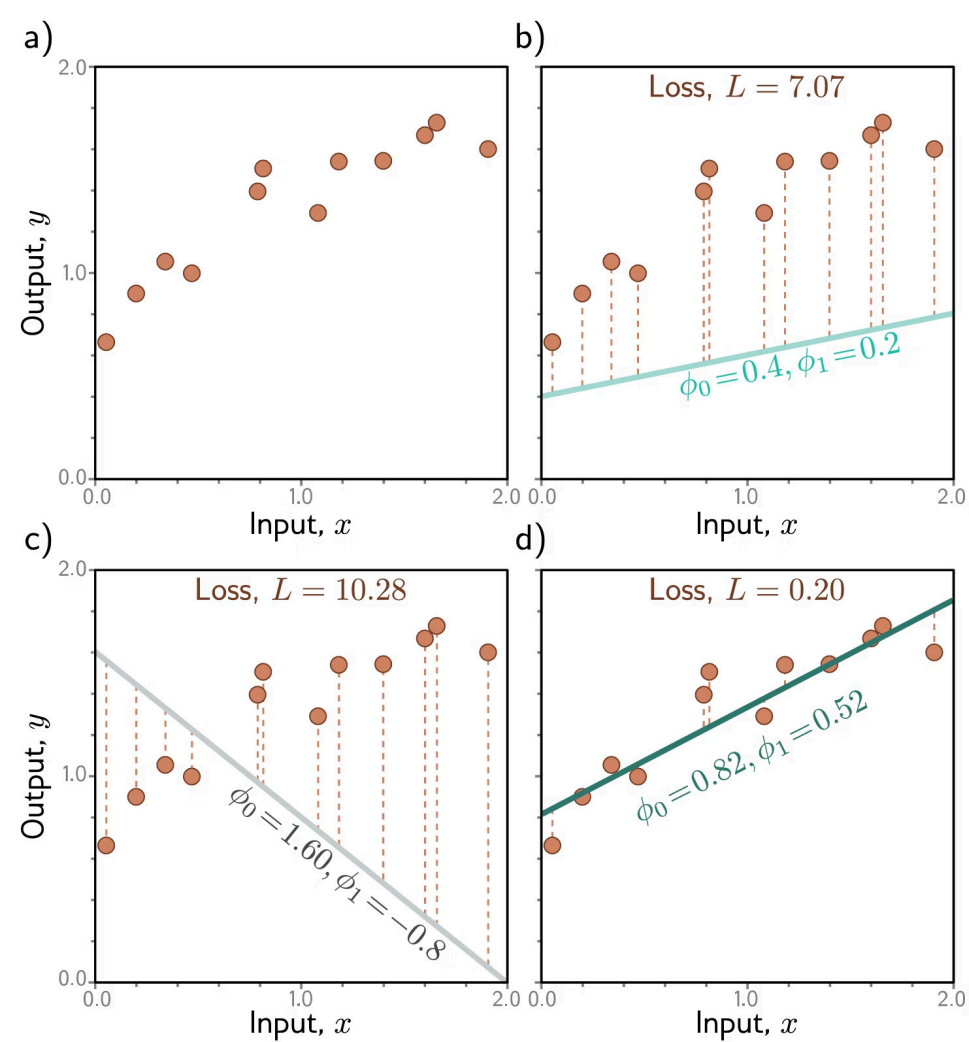


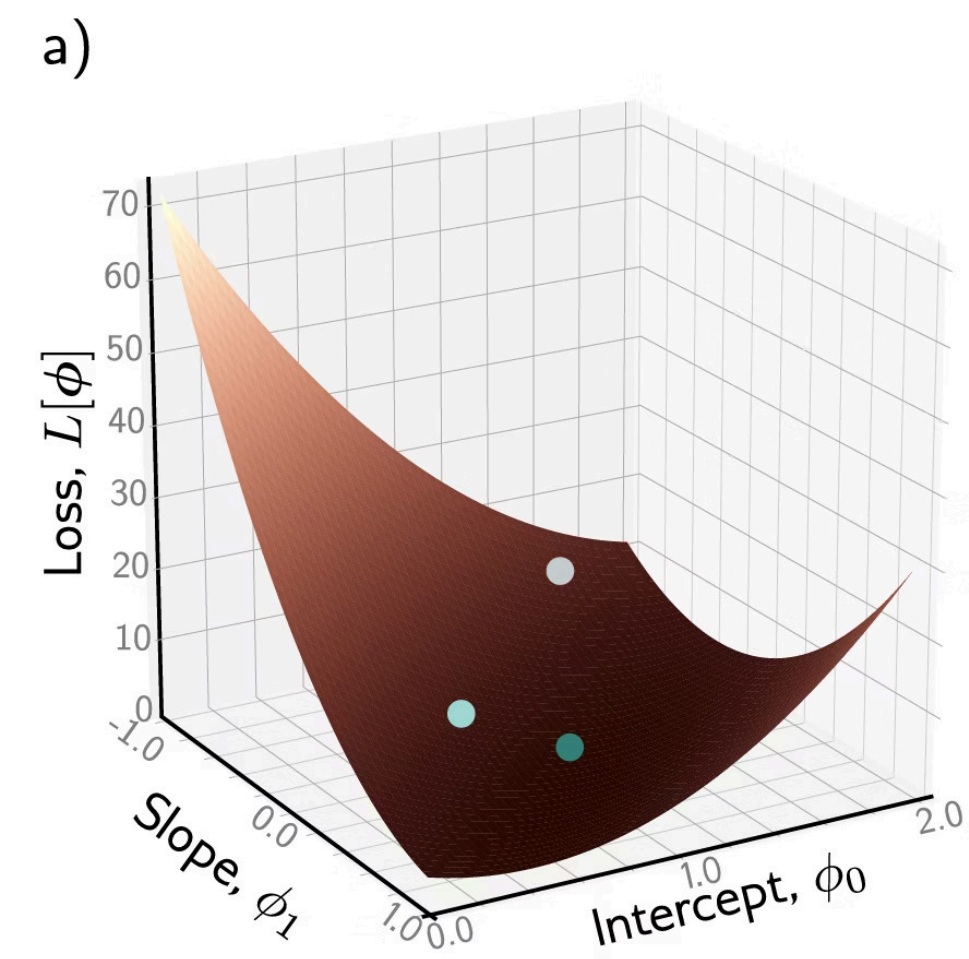
Figura 2.2

Definiamo la loss function come (SSE = Sum of Squared Errors):

$$L[\phi] = \sum_{i=1}^I (\mathbf{f}[x_i, \phi] - y_i)^2 = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

L'obiettivo dell'apprendimento è quindi trovare un valore per i parametri tale che:

$$\begin{aligned} \hat{\phi} &= \arg \min_{\phi} [L[\phi]] \\ &= \arg \min_{\phi} \left[\sum_{i=1}^I (\mathbf{f}[x_i, \phi] - y_i)^2 \right] \\ &= \arg \min_{\phi} \left[\sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \right] \end{aligned}$$



Ad ogni combinazione di parametri

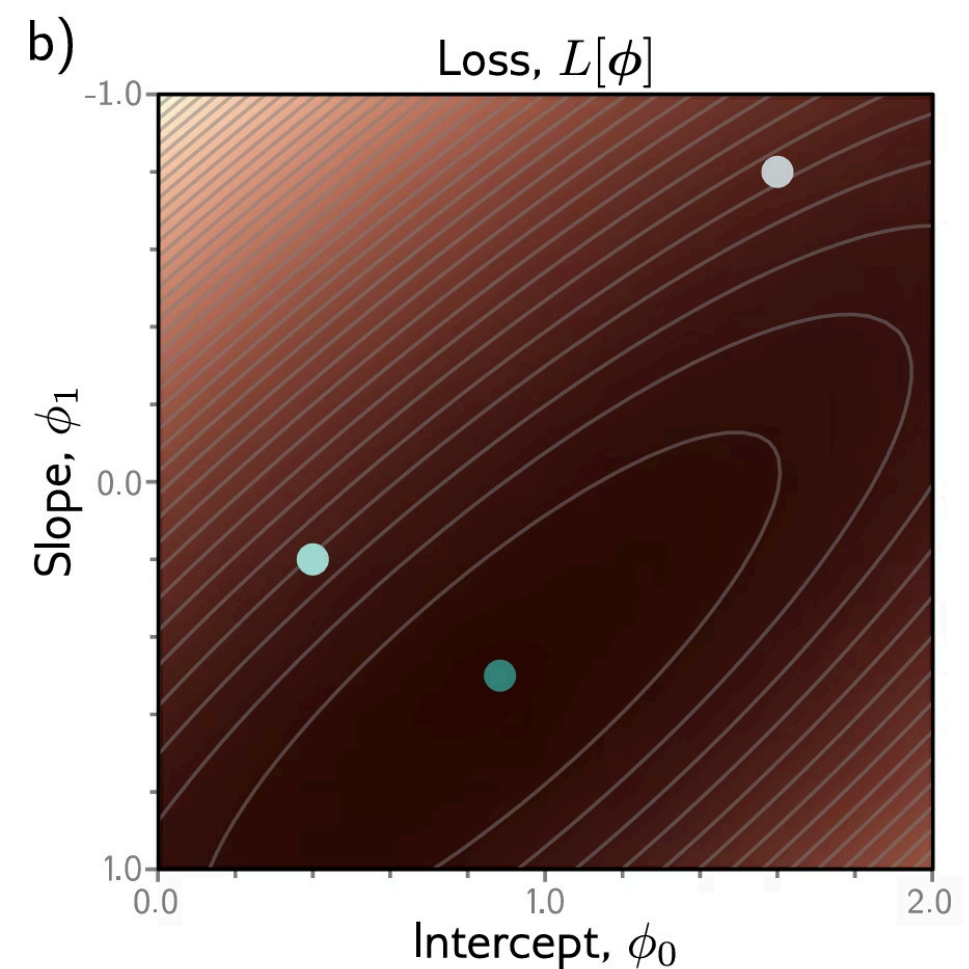
$$\phi = [\phi_0, \phi_1]$$

corrisponde un valore di loss.

La loss function

$$L[\phi]$$

può quindi essere rappresentata come una superficie.



La perdita può anche essere visualizzata come una mappa di calore, dove le regioni più luminose rappresentano perdite maggiori.

La perdita più piccola è il cerchio verde.

Shallow Neural Network

Si consideri la seguente funzione scalare

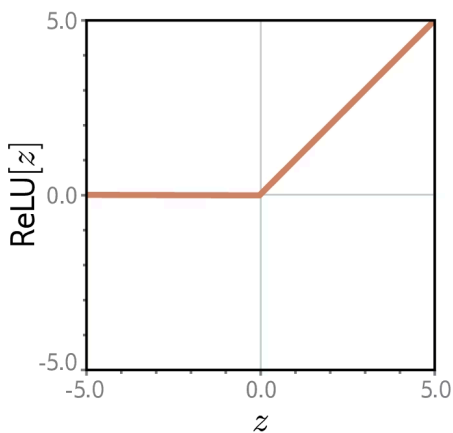
$$y = \mathbf{f}[x, \boldsymbol{\phi}]$$
$$= \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x]$$

che mappa un valore scalare x in un valore scalare y e che dipende dai parametri

$$\boldsymbol{\phi} = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$$

a , **funzione di attivazione** è definita come segue:

$$a[z] = \text{ReLU}[z]$$
$$= \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$



Questa funzione di attivazione si chiama **Rectified Linear Unit (ReLU)**. ReLU è una funzione **non lineare**.

Scomponiamo il precedente calcolo nel seguente modo:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$
$$h_2 = a[\theta_{20} + \theta_{21}x]$$
$$h_3 = a[\theta_{30} + \theta_{31}x]$$

h_1, h_2, h_3 sono definite **hidden units**.

Combiniamo quindi i risultati intermedi per calcolare il valore finale:

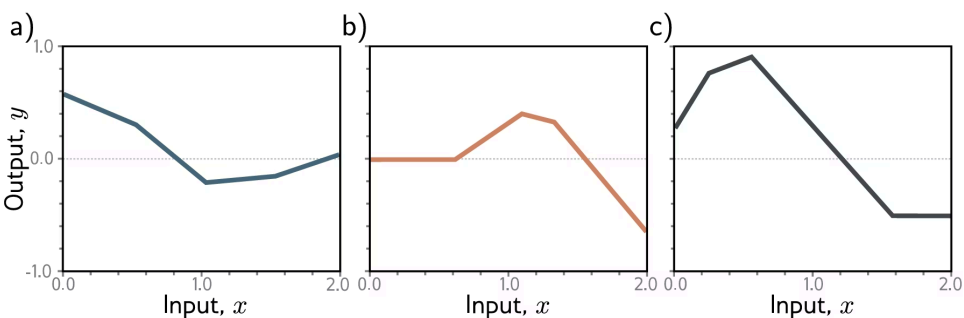
$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Struttura delle Unità Nascoste

- Ogni unità nascosta calcola una funzione lineare

$$\theta_{\bullet 0} + \theta_{\bullet 1}x$$

- La funzione viene troncata da ReLU:
 - Se il valore è negativo → clipping a zero.
 - Se il valore è positivo → attivazione della funzione.
- I punti in cui le rette attraversano zero diventano i giunti della funzione finale.



Funzioni per tre diverse scelte dei dieci parametri. In ogni caso, la relazione input/output è lineare a tratti. Tuttavia, le posizioni dei giunti, le pendenze delle regioni lineari tra di loro e l'altezza complessiva variano.

Combinazione e Peso delle Unità Attive

- Le tre linee troncate vengono moltiplicate per i pesi ϕ_1, ϕ_2, ϕ_3 .
- Il termine ϕ_0 viene aggiunto per regolare l'altezza complessiva della funzione risultante.

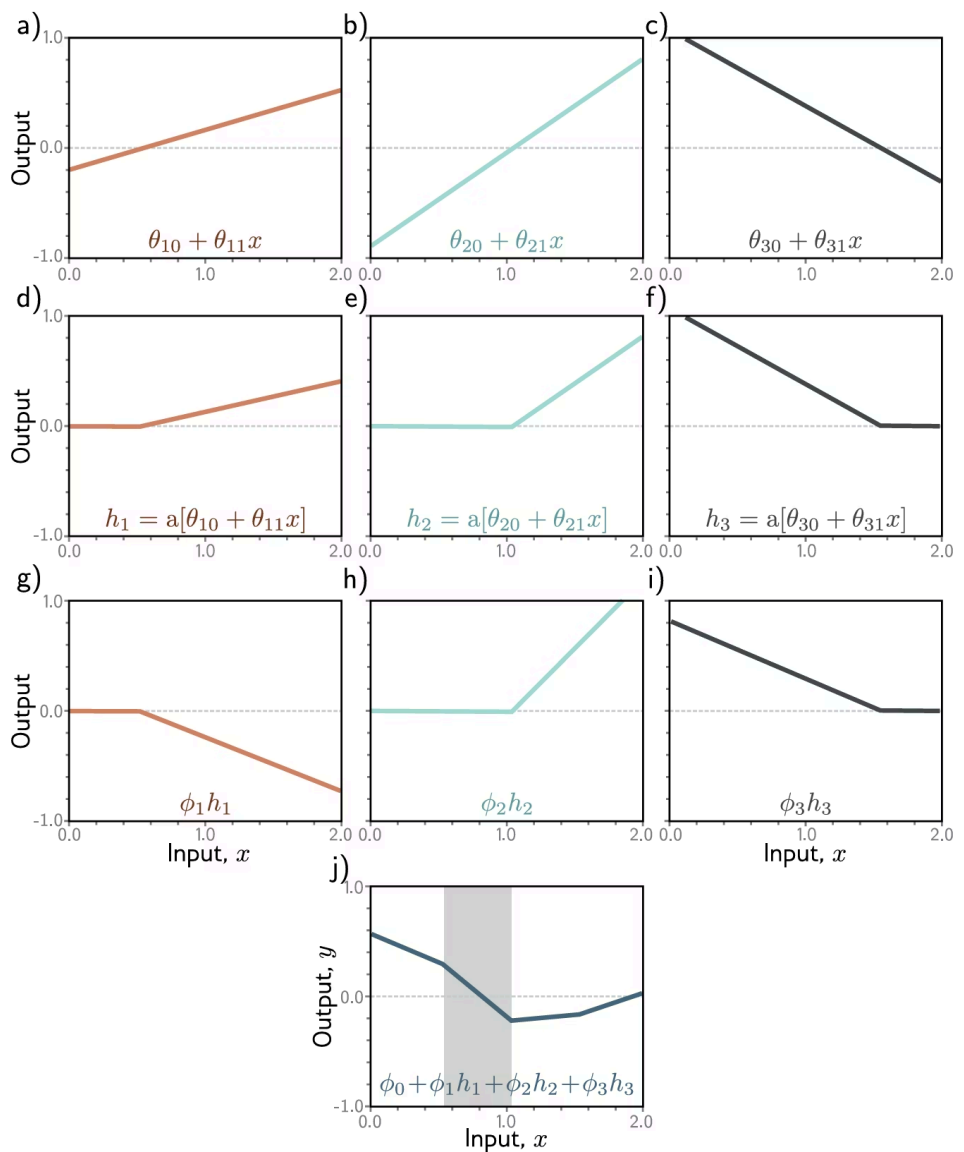
Attivazione e Inattivazione delle Unità Nascoste

- Un'unità è attiva** quando il suo output non viene troncato dalla ReLU.
- Un'unità è inattiva** quando il suo output è stato troncato a zero.
- Ogni regione della funzione finale corrisponde a un **diverso pattern di attivazione**.
- Esempio (regione ombreggiata a destra):
 - h_1 e h_3 sono attivi, h_2 è inattivo
 - Il coefficiente angolare (**slope**) nella zona grigia è dato dalla somma dei contributi attivi:

$$slope = \phi_1 h_1 + \phi_3 h_3$$

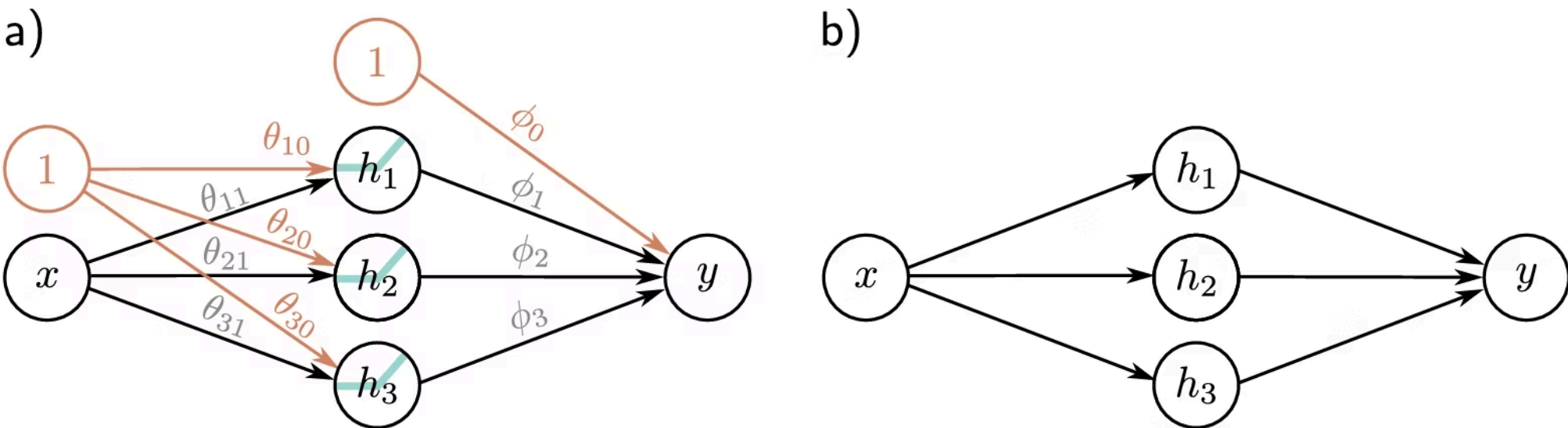
Numero di Regioni Lineari

- Con **3 unità nascoste**, la funzione può avere fino a **4 regioni lineari**.



3.3a - 1D shallow network (ReLU)

Quanto mostrato sopra è l'esempio di una **shallow neural network**, che tipicamente viene rappresentata nel seguente modo:



I disegni mostrano una rete neurale non profonda con un input, un output e uno strato nascosto.

Teorema di Approssimazione Universale

Generalizziamo la precedente rete immaginando uno strato nascosto composto da D hidden units. La d -esima hidden unit è definita come:

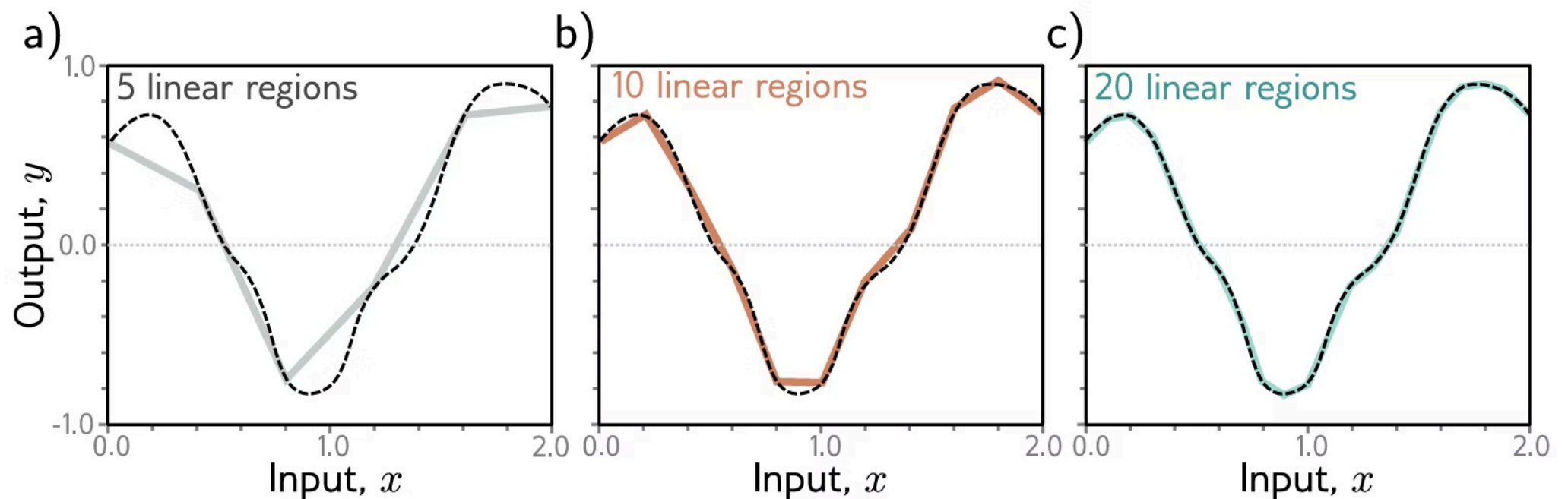
$$h_d = a[\theta_{d0} + \theta_{d1}x]$$

Le hidden unit sono combinate per calcolare il valore finale:

$$y = \phi_0 + \sum_{d=1}^D \phi_d h_d$$

Il numero di hidden unit è una misura della capacità della rete.

Con le funzioni di attivazione ReLU, l'output di una rete con D unità nascoste ha al massimo D giunzioni e quindi è una funzione lineare a tratti con al massimo $D + 1$ regioni lineari. Man mano che aggiungiamo più unità nascoste, il modello può approssimare funzioni più complesse.



- ❑ Il **Teorema di Approssimazione Universale** afferma che per qualsiasi funzione continua definita su un dominio chiuso e limitato, esiste una rete neurale con un singolo strato nascosto e un numero sufficiente di neuroni che può approssimare quella funzione con qualsiasi precisione desiderata.

Va tenuto presente che si tratta di un risultato di esistenza: ci dice che una tale rete esiste, ma non come costruirla né quanti neuroni servano in pratica.

Output Multivariato (Multidimensionale)

- Finora abbiamo considerato reti con **un solo output scalare**.
- Per estendere il modello ad un **output multivariato** (con y che non è più un valore scalare ma un vettore di scalari), basta definire una funzione lineare separata per ogni componente dell'output.
- Supponendo ad esempio di avere una rete con un input scalare, 4 hidden unit e 2 output (2D vector) $y = [y_1, y_2]^T$, definiamo:

$$h_1 = a [\theta_{10} + \theta_{11}x]$$

$$h_2 = a [\theta_{20} + \theta_{21}x]$$

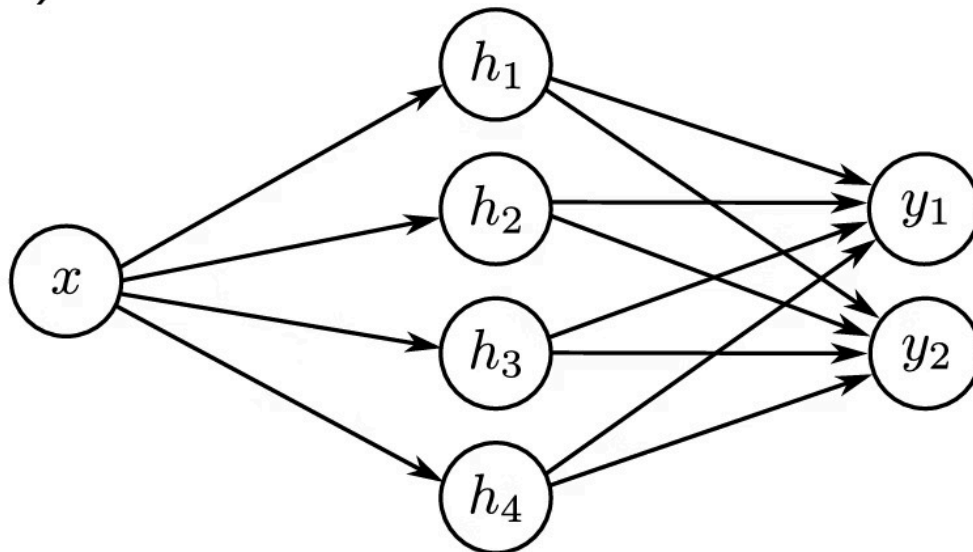
$$h_3 = a [\theta_{30} + \theta_{31}x]$$

$$h_4 = a [\theta_{40} + \theta_{41}x]$$

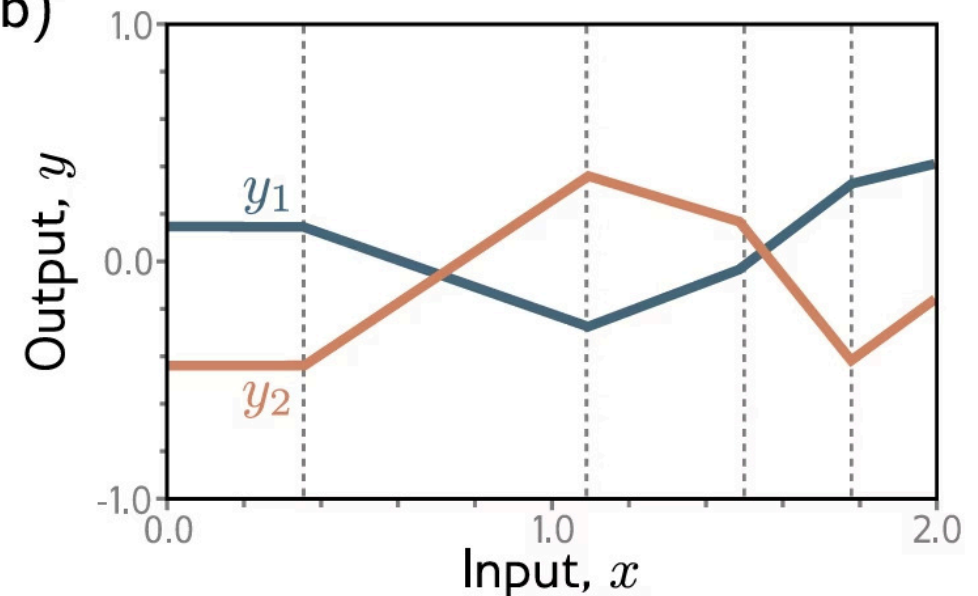
$$y_1 = \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4$$

$$y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$$

a)

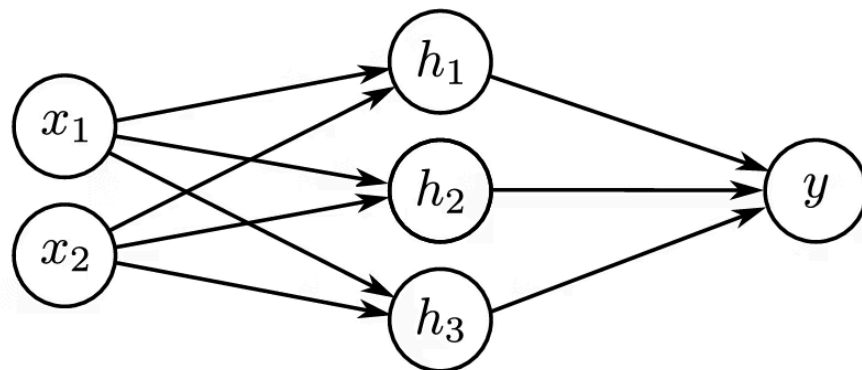


b)



Input Multivariato (Multidimensionale)

Definiamo ora il caso di una rete che ha un input multivariato, ovvero un vettore composto da più scalari (supponiamo per semplicità due) $x = [x_1, x_2]^T$ e un output scalare y e 3 hidden unit:



$$h_1 = a [\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2]$$

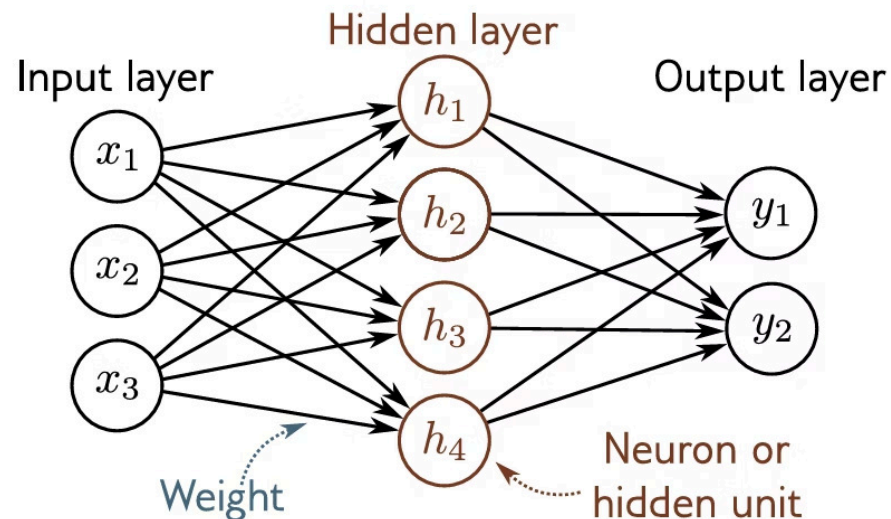
$$h_2 = a [\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2]$$

$$h_3 = a [\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2]$$

$$y = \phi_0 + \phi_1h_1 + \phi_2h_2 + \phi_3h_3$$

Il caso generale

Consideriamo ora il caso di una rete shallow che mappa un input multidimensionale $\mathbf{X} \in \mathbb{R}^{D_i}$ in un output multidimensionale $\mathbf{Y} \in \mathbb{R}^{D_o}$ attraverso $\mathbf{h} \in \mathbb{R}^D$ hidden unit:



Ogni hidden unit è calcolata come segue:

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^D \theta_{di} x_i \right]$$

queste vengono combinate linearmente per calcolare l'output finale:

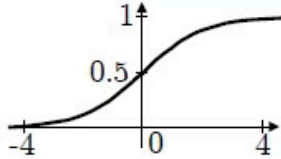
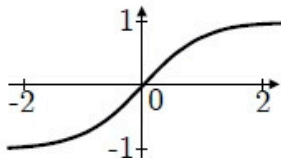
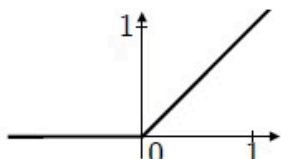
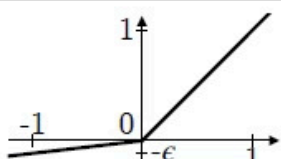
$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d$$

⚠ **La funzione di attivazione consente al modello di descrivere relazioni non lineari tra input e output e, in quanto tale, deve essere essa stessa non lineare; senza una funzione di attivazione o una funzione di attivazione lineare, la mappatura complessiva da input a output sarebbe limitata a essere lineare.**

- **Multi-Layer Perceptron (MLP):** qualsiasi rete con almeno **un hidden layer**.
- **Shallow Neural Network:** rete con **un solo hidden layer**.
- **Deep Neural Network (DNN):** rete con **più hidden layers**.
- **Feed-Forward Network:**
 - Le connessioni formano un **grafo aciclico (senza loop)**.
 - Ogni strato riceve input solo dallo strato precedente.
- **Fully Connected Network:**
 - Ogni neurone è **connesso a tutti i neuroni** dello strato successivo.

Funzioni di attivazione

ReLU è la più comune funzione di attivazione ma non è l'unica.

Name	Function $\mathcal{A}(z)$	Output range	Illustration
Sigmoid σ <i>Sigmoid function</i>	$\frac{1}{1 + e^{-z}}$	$]0, 1[$	
Tanh <i>Hyperbolic Tangent</i>	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$] - 1, 1[$	
ReLU <i>Rectified Linear Unit</i>	$\max(0, z)$	$[0, +\infty[$	
Leaky ReLU <i>Leaky Rectified Linear Unit</i>	$\max(\epsilon z, z)$ with $\epsilon \ll 1$	$] - \infty, +\infty[$	

[3.3b - 1D shallow network \(sigmoid\)](#)

GELU – Gaussian Error Linear Unit

- ❏ Le architetture Transformer richiedono funzioni di attivazione che facilitino il flusso del gradiente attraverso reti molto profonde. ReLU, con il suo taglio netto a zero, introduce discontinuità nel gradiente che possono ostacolare l'ottimizzazione. GELU è stata introdotta come alternativa più morbida ed è diventata la funzione di attivazione standard in BERT, GPT e Vision Transformer.

A differenza di ReLU, che decide in modo netto se attivare o spegnere un neurone, GELU lo fa in modo probabilistico: moltiplica l'input per la probabilità che sia maggiore di zero secondo una distribuzione gaussiana standard.

$$GELU(x) = x \cdot \Phi(x)$$

dove $\Phi(x)$ è la CDF (Cumulative Distribution Function) della distribuzione normale standard.

Comportamento

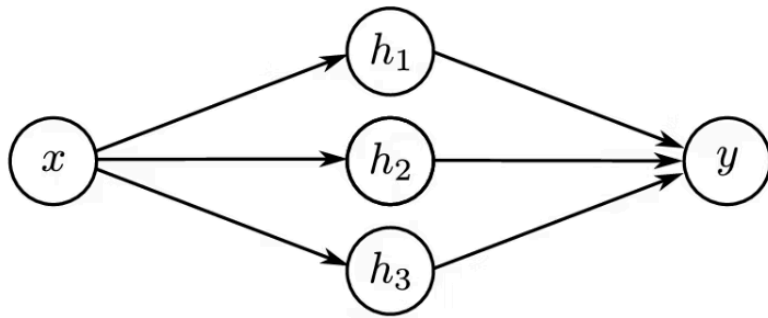
- **x molto positivo** $\rightarrow \Phi(x) \approx 1 \rightarrow \text{output} \approx x$ (il segnale passa)
- **x molto negativo** $\rightarrow \Phi(x) \approx 0 \rightarrow \text{output} \approx 0$ (il segnale è soppresso)
- **x vicino a zero** $\rightarrow$ transizione graduale e morbida

Perché è importante

- Nessuna discontinuità nel gradiente (a differenza del "taglio netto" di ReLU in $x = 0$)
- Leggermente non-monotona: piccoli input negativi non vengono completamente azzerati
- Funzione di attivazione standard nei Transformer (BERT, GPT, ViT)

L'approssimazione pratica comunemente usata è: $GELU(x) \approx x \cdot \sigma(1.702x)$, dove σ è la funzione sigmoide.

Notazione Matriciale



Riprendiamo la rete shallow vista in precedenza (con un input scalare x , 3 hidden unit e un output scalare y) e riscriviamola in forma matriciale.

Le equazioni che già conosciamo:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

Possono essere riscritte in notazione matriciale come:

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \mathbf{a} \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right)$$

$$y = \phi_0 + \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}$$

In forma compatta:

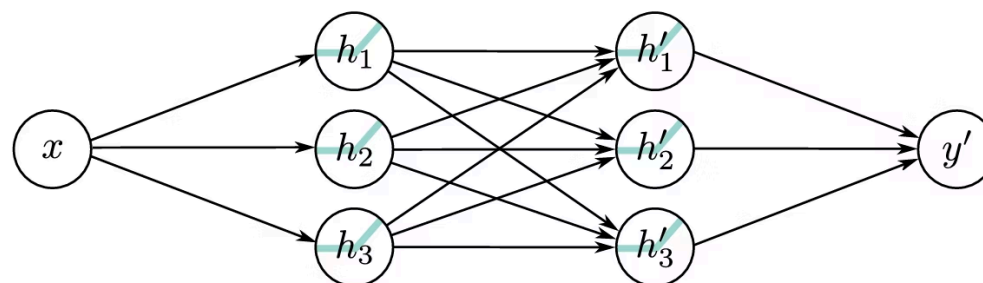
$$\mathbf{h} = \mathbf{a}[\boldsymbol{\theta}_0 + \boldsymbol{\theta}_1 x]$$

$$y = \phi_0 + \boldsymbol{\phi}^T \mathbf{h}$$

- ❑ Perché la notazione matriciale? Non è solo una comodità formale: le operazioni matriciali sono ciò che rende l'addestramento computazionalmente fattibile. Le GPU sono ottimizzate per la moltiplicazione di matrici, e questa rappresentazione permette di elaborare interi batch di dati in parallelo.

Notazione Matriciale – Estensione alla Rete Profonda

Estendiamo la notazione matriciale a una rete con due hidden layer. Lo schema della rete prevede un input scalare x , un primo strato nascosto con 3 unità h_1, h_2, h_3 , un secondo strato nascosto con 3 unità h'_1, h'_2, h'_3 e un output y' .



Il primo hidden layer (identico al caso shallow):

$$\begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = \mathbf{a} \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right)$$

Il secondo hidden layer: ogni unità riceve in input l'intero vettore $\mathbf{h}$ dallo strato precedente, quindi la matrice dei pesi diventa quadrata:

$$\begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix} = \mathbf{a} \left(\begin{bmatrix} \psi_{10} \\ \psi_{20} \\ \psi_{30} \end{bmatrix} + \begin{bmatrix} \psi_{11} & \psi_{12} & \psi_{13} \\ \psi_{21} & \psi_{22} & \psi_{23} \\ \psi_{31} & \psi_{32} & \psi_{33} \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \right)$$

L'output finale:

$$y' = \phi'_0 + \begin{bmatrix} \phi'_1 & \phi'_2 & \phi'_3 \end{bmatrix} \begin{bmatrix} h'_1 \\ h'_2 \\ h'_3 \end{bmatrix}$$

In forma compatta, l'intera rete si descrive in tre righe:

$$\mathbf{h} = \mathbf{a}[\boldsymbol{\theta}_0 + \boldsymbol{\theta}x]$$

$$\mathbf{h}' = \mathbf{a}[\boldsymbol{\psi}_0 + \boldsymbol{\Psi}\mathbf{h}]$$

$$y' = \phi'_0 + \boldsymbol{\phi}'^T \mathbf{h}'$$

- Il pattern è ricorsivo – ogni strato applica la stessa struttura: combinazione lineare (matrice dei pesi $\times$ input + bias) seguita da una funzione di attivazione non lineare. Aggiungere strati significa semplicemente ripetere questo schema. Questa regolarità è ciò che rende le reti neurali profonde eleganti dal punto di vista computazionale e facili da implementare in codice.

Broadcasting

Quando elaboriamo un intero batch di m esempi contemporaneamente, l'operazione di uno strato di una rete neurale assume la seguente forma matriciale:

$$Z = XW + \mathbf{b}$$

Dove:

- $X \in \mathbb{R}^{m \times n}$ (ovvero m esempi con n feature ciascuno)
- $W \in \mathbb{R}^{n \times p}$, W è la matrice dei pesi
- $\mathbf{b} \in \mathbb{R}^{1 \times p}$, $\mathbf{b}$ è il vettore dei bias
- $Z \in \mathbb{R}^{m \times p}$
- $S = XW \in \mathbb{R}^{m \times p}$

Come faccio a sommare la matrice $XW \in \mathbb{R}^{m \times p}$ e il vettore riga $\mathbf{b} \in \mathbb{R}^{1 \times p}$?

Il meccanismo di **broadcasting** risolve questo problema replicando automaticamente il vettore $\mathbf{b}$ lungo la dimensione del batch, come se diventasse una matrice con dimensioni $(m \times p)$:

$$\begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix} \longrightarrow \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{21} & s_{22} & s_{23} \end{bmatrix} + \begin{bmatrix} b_1 & b_2 & b_3 \\ b_1 & b_2 & b_3 \end{bmatrix}$$

In pratica, lo stesso bias viene sommato a ogni esempio del batch. Questo meccanismo è gestito automaticamente da librerie di deep learning come NumPy e PyTorch.

- ❏ Il broadcasting non è solo una comodità: è il motivo per cui la notazione matriciale è direttamente traducibile in codice efficiente, ottimizzato per l'elaborazione parallela su GPU, rendendo l'addestramento delle reti neurali computazionalmente fattibile su larga scala.

Shallow vs Deep Neural Networks

Efficienza della Profondità

- **Sia le reti profonde che quelle superficiali possono approssimare funzioni arbitrarie.**
- Tuttavia, **alcune funzioni richiedono un numero esponenzialmente maggiore di unità nascoste in una rete superficiale** rispetto a una rete profonda per ottenere lo stesso livello di approssimazione.
- Questo fenomeno è noto come [depth efficiency](#) delle reti neurali.
- **Limite pratico:** non è sempre chiaro se le funzioni reali rientrano in questa categoria, quindi la scelta della profondità del modello dipende dal problema specifico.

Limitazioni delle Reti Completamente Connesse (Fully Connected Networks)

- **Problema:** per input strutturati molto grandi (es. immagini con $\sim 10^6$ pixel), una rete completamente connessa sarebbe **computazionalmente proibitiva**.
- **Motivazione:**
 - Ogni pixel sarebbe connesso a tutti i neuroni del livello successivo → **troppe connessioni** e parametri da apprendere.
 - Inoltre, vogliamo che la rete **riconosca gli stessi oggetti ovunque nell'immagine**, senza doverli apprendere indipendentemente in ogni posizione.
- **Soluzione:** elaborare **regioni locali dell'immagine in parallelo** e poi combinare l'informazione in livelli successivi.
- **Importanza della profondità:** questo approccio **richiede più strati** per integrare gradualmente l'informazione locale fino a una comprensione globale dell'immagine.

Addestramento e Generalizzazione delle Reti Profonde

- **Facilità di Addestramento:**
 - **Moderatamente profonde:** spesso **più facili da addestrare** rispetto alle reti superficiali, perché hanno più parametri e quindi una maggiore varietà di soluzioni possibili.
 - **Molto profonde:** l'addestramento diventa **più difficile** a causa di problemi come il [vanishing gradient](#), ma esistono tecniche per mitigarli.
- **Generalizzazione:**
 - Le **reti profonde sembrano generalizzare meglio** rispetto a quelle superficiali.
 - In pratica, i migliori risultati su molteplici task sono stati ottenuti con reti **di decine o centinaia di strati**.
 - **Tuttavia, la ragione teorica di questo fenomeno non è ancora del tutto compresa.**

In Sintesi

- **Le reti profonde sono più efficienti per alcune classi di funzioni**, riducendo il numero di unità necessarie rispetto alle reti shallow.
- **Le reti profonde generalizzano meglio e possono essere più facili da addestrare**, ma l'**aumento della profondità introduce nuove sfide**.

Addestramento di un Modello

- Per addestrare un modello serve un training set composto da coppie input/output $\{x_i, y_i\}$.
- L'obiettivo è trovare i parametri ϕ , del modello $f[x_i, \phi]$, che mappano al meglio gli input negli output.
- Si definisce a priori una loss function $L[\phi]$ che quantifica l'errore di mapping.
- L'obiettivo dell'addestramento è trovare i parametri $\hat{\phi}$ che minimizzano la loss:

$$\hat{\phi} = \arg \min_{\phi} L[\phi]$$

- Esistono diverse famiglie di algoritmi di **ottimizzazione** atte a minimizzare la perdita complessiva.
- Per l'addestramento di modelli si utilizzano metodi **iterativi**:
 - I parametri vengono inizializzati **euristicamente**.
 - Si calcolano le derivate (gradienti) della perdita rispetto ai parametri
 - I parametri vengono poi aggiornati nella direzione che riduce la perdita fino a quando la perdita è zero o sotto una certa soglia prestabilita.
- Il metodo più semplice e diffuso è la **discesa del gradiente (gradient descend)**.

Il Gradiente

- Il **gradiente** è un vettore che raccoglie tutte le derivate parziali della loss function rispetto a ogni singolo parametro.
- Il gradiente punta nella direzione in cui la loss function subisce l'incremento maggiore.
- Siccome ci interessa minimizzare la loss function, **dobbiamo andare nella direzione opposta.**
- Se il gradiente diventa zero abbiamo raggiunto un minimo locale.

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}$$

Ogni singola derivata parziale è una misura di sensibilità isolata, ovvero indica quanto varia L al variare di ϕ_i .

$$\frac{\partial L}{\partial \phi_i}$$

La variazione della loss function ΔL , in risposta a una variazione infinitesima di un singolo parametro $\Delta \phi_i$, è approssimata dal prodotto tra il gradiente (la derivata parziale) e la variazione del parametro stesso.

$$\Delta L \approx \frac{\partial L}{\partial \phi_i} \cdot \Delta \phi_i$$

Ad esempio se la derivata di L rispetto ad ϕ_i è 4.0, significa che se incrementiamo ϕ_i di una **quantità piccola** (ad esempio $\Delta \phi_i = 0.001$), L subirà un incremento di circa 4 volte quella quantità, risultando in un $\Delta L = 0.004$

Learning Rate

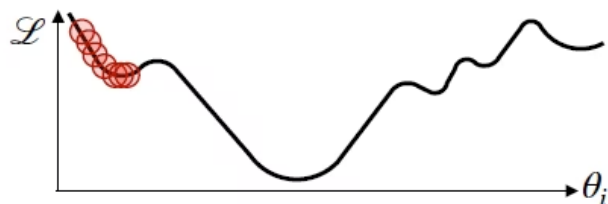
Per garantire che l'algoritmo operi sempre nello "Scenario A" (dove l'approssimazione locale regge), dobbiamo vincolare l'aggiornamento. Invece di sottrarre direttamente l'intero gradiente o un valore arbitrario, lo ridimensioniamo sistematicamente.

Questo fattore di scala è il **Learning Rate** (α), portando all'equazione standard dell'ottimizzazione:

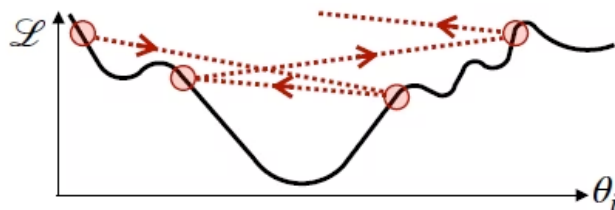
$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}$$

Il Learning Rate è un **iper-parametro**: non è sottoposto ad apprendimento ma va definito a priori.

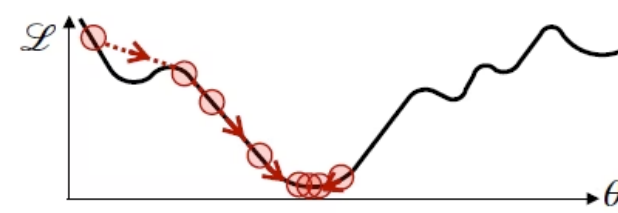
Troppo basso, si rischia di convergere troppo lentamente o di stazionare in un minimo locale.



Troppo alto, si velocizza il processo di convergenza, tuttavia si rischia di divergere e quindi non individuare un minimo.



Un valore corretto di α consente alla rete di convergere verso un punto ottimale.



Discesa del Gradiente

- Si inizializzano i parametri $\phi = [\phi_0, \phi_1, \dots, \phi_N]^T$ con un valore iniziale (ad es. random).
- Ripeti per un numero predefinito di step oppure quando il valore di loss è sotto una certa soglia predefinita:
 - **(Step 1)** Si calcola la derivata prima della loss function rispetto a ciascun parametro ϕ_i ; l'insieme di tutte le derivate parziali compone un vettore detto appunto **gradiente**

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_N} \end{bmatrix}$$

- **(Step 2)** Si aggiorna ogni singolo parametro secondo la seguente regola:

$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}$$

α è un valore scalare positivo che determina la magnitudo del cambiamento (**learning rate**)

Chain Rule



- Se un'automobile viaggia a una velocità doppia rispetto a una bicicletta.
- E se la bicicletta viaggia a una velocità quattro volte superiore rispetto a un uomo che cammina.
- Quanto viaggia più veloce l'automobile rispetto all'uomo?

La risposta intuitiva è il prodotto dei due tassi: $2 \cdot 4 = 8$ volte più veloce.

Questa semplice moltiplicazione dei tassi di variazione intermedi è l'essenza stessa della **chain rule**.



Se una variabile z dipende da una variabile intermedia y , e y a sua volta dipende da x , la derivata di z rispetto a x si ottiene moltiplicando le due derivate parziali:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Per spiegare come questo si applica alle reti neurali, dobbiamo scomporre il processo in moduli logici che operano su un **grafo computazionale**:

- Visione Locale:** Una rete neurale, per quanto profonda, è solo una gigantesca espressione matematica scomponibile in operazioni elementari (addizioni, moltiplicazioni). Ogni operazione è un piccolo "nodo" indipendente che conosce solo i propri input e il proprio output.
- Il Gradiente Locale:** Ogni nodo sa perfettamente come calcolare la sua derivata locale (es. come l'output cambia al variare del suo specifico input).
- Il Gradiente Globale:** Durante la fase di backpropagation, il nodo riceve dal suo "genitore" il gradiente globale, ovvero la derivata della Loss finale rispetto all'output di quel nodo.
- La Moltiplicazione:** Il nodo applica la regola della catena moltiplicando semplicemente il gradiente globale ricevuto per il proprio gradiente locale, passando poi il risultato a ritroso ai suoi figli.

Applicazione Pratica: Scomponiamo il nostro Neurone ReLU

Riprendiamo esattamente il modello che abbiamo costruito in precedenza e applichiamo la regola della catena per calcolare $\frac{\partial L}{\partial w}$. Il flusso dei dati all'indietro attraversa tre passaggi:

A. Il Nodo della Loss (Mean Squared Error)

- Formula: $L = (\hat{y} - y_{true})^2$
- La rete ci fornisce il gradiente globale partendo dalla fine. La derivata locale di questa operazione rispetto alla previsione $\hat{y}$ è semplicemente la derivata di un quadrato:
- $\frac{\partial L}{\partial \hat{y}} = 2 \cdot (\hat{y} - y_{true})$

B. Il Nodo di Attivazione (ReLU)

- Formula: $\hat{y} = \max(0, n)$
- Il suo gradiente locale è 1 se l'input n è maggiore di 0, altrimenti è 0.
- $\frac{\partial \hat{y}}{\partial n} = 1$ (assumendo che il neurone sia attivo).

C. Il Nodo della Moltiplicazione (Il Peso)

- Formula: $n = w \cdot x + b$
- La derivata locale di una moltiplicazione $w \cdot x$ rispetto a w è semplicemente l'altro termine, ovvero x .
- $\frac{\partial n}{\partial w} = x$

La Sintesi della Chain Rule:

Ora uniamo i pezzi del puzzle moltiplicando i tassi di variazione lungo il percorso, proprio come nell'esempio dell'automobile:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial n} \cdot \frac{\partial n}{\partial w}$$

$$\frac{\partial L}{\partial w} = [2 \cdot (\hat{y} - y_{true})] \cdot [1] \cdot [x]$$

Backpropagation

- **Backpropagation**, utilizzato in combinazione con un metodo di ottimizzazione come la discesa del gradiente, è l'algoritmo più diffuso (uno standard de-facto) per l'addestramento di reti neurali.
- Paul Werbos è stato il primo ad introdurre l'idea nel 1974.
- [Geoffrey Hinton](#), **David E. Rumelhart** e **Ronald J. Williams**, nel 1986 hanno pubblicato l'articolo "[Learning representations by back-propagating errors](#)", rendendolo popolare e dimostrandone l'efficacia.

Algoritmo

- Inizializza i pesi e i bias della rete con valori casuali piccoli
- Per ogni iterazione dell'addestramento:
 - **Forward Propagation**
Propaga l'input $\mathbf{x}$ attraverso la rete per ottenere $\mathbf{f}[\mathbf{x}, \boldsymbol{\phi}]$
 - **Calcolo dell'errore**
Calcola la perdita totale L confrontando l'output finale $\mathbf{f}[\mathbf{x}, \boldsymbol{\phi}]$ con quello proveniente dal training set $\mathbf{y}$
 - **Backward Propagation**
 - Calcola il gradiente della perdita rispetto all'output finale
 - Propaga il gradiente all'indietro attraverso la rete. Per ogni strato:
 - Calcola il gradiente rispetto a pesi e bias
 - Calcola il gradiente rispetto agli output dello strato precedente
 - Aggiornamento dei pesi e bias utilizzando la discesa del gradiente:
 - $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}}$
 - $\mathbf{b} \leftarrow \mathbf{b} - \alpha \cdot \frac{\partial L}{\partial \mathbf{b}}$
 - Ripeti fino a convergenza o fino al numero massimo di iterazioni.

Adam (Adaptive Moment Estimation)

- A differenza della semplice discesa del gradiente (che sottrae passivamente il gradiente moltiplicato per il learning rate), Adam è più sofisticato. Il suo segreto risiede nel mantenere due "medie mobili" storiche per ogni singolo parametro del modello.
- **L'Inerzia (Il "Momentum" - buffer m)**
 - **Cosa fa:** Calcola e ricorda la media dei gradienti più recenti.
 - **L'Analogia:** Si comporta esattamente come una palla che rotola lungo un pendio. Accumula velocità nella direzione in cui sta scendendo, aiutando l'algoritmo a non bloccarsi nelle piccole irregolarità della funzione di Loss.
- **Il Passo Adattivo (La "Varianza" - buffer v)**
 - **Cosa fa:** Calcola la media dei quadrati dei gradienti recenti.
 - **L'Analogia:** Agisce come un "freno/acceleratore" intelligente. Permette ad Adam di adattare dinamicamente il *learning rate* in modo specifico per ogni singolo parametro. Se un peso subisce variazioni violente, Adam ne riduce il passo; se varia poco, lo accelera.

Batch ed Epochs

Introduzione della Casualità

- Ad ogni iterazione, l'algoritmo sceglie un **sottoinsieme casuale** dei dati di training (**minibatch** o **batch**).
- Il gradiente viene calcolato solo su questo sottoinsieme, invece che sull'intero dataset.

Regola di Aggiornamento

- I parametri del modello vengono aggiornati secondo la formula:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in B_t} \frac{\partial \ell_i(\phi_t)}{\partial \phi}$$

Dove:

- B_t è l'insieme degli indici del batch corrente.
- $\partial \ell_i$ è la loss relativa alla coppia (x_i, y_i)
- α è il **learning rate**, che determina l'entità dell'aggiornamento.

Definizione di Epoch

- I batch vengono scelti **senza ripetizione** fino a completare l'intero dataset.
- **Un'epoca (epoch)** corrisponde a un passaggio completo su tutti i dati di training.
- Il batch può variare in grandezza:
 - **Batch Gradient Descent** — un solo "batch" che coincide con l'intero dataset. Si fa un unico aggiornamento per epoca. Più stabile ma lento e costoso in memoria.
 - **Stochastic Gradient Descent (SGD puro)** — batch size = 1. Si aggiorna dopo ogni singolo esempio. Molto rumoroso ma veloce.
 - **Mini-Batch Gradient Descent**
 - All'inizio di ogni epoca, il dataset di training viene rimescolato casualmente (shuffle)
 - Il dataset rimescolato viene suddiviso in blocchi sequenziali di dimensione fissa, detti mini-batch
 - Per ciascun mini-batch si eseguono in sequenza: forward pass → calcolo della loss → backward pass → aggiornamento dei pesi
 - Quando tutti i mini-batch sono stati processati, ogni esempio è stato visto esattamente una volta: l'epoca è completa
 - Si rimescolano i dati e si ricomincia con l'epoca successiva
 - **Perché "stocastico"?**
 - Il gradiente calcolato su un mini-batch è una stima approssimata del gradiente vero (calcolato sull'intero dataset)
 - Questa approssimazione introduce rumore, che aiuta a sfuggire da minimi locali e punti di sella
 - Il compromesso usato in pratica, dimensionato in base alla memoria GPU disponibile

Modelli non lineari

- **Funzioni di Loss nella Regressione Lineare**

- Le funzioni di loss per la regressione lineare hanno un **unico minimo globale ben definito**.
- Questo perché sono **convesse**.

- **Proprietà della Convessità**

- La **convessità** garantisce che, indipendentemente dall'inizializzazione dei parametri, il minimo sarà sempre raggiunto.
- Basta **seguire la direzione della discesa del gradiente** per ottenere la soluzione ottimale.
- Con un learning rate appropriato, il training converge sempre al minimo globale.

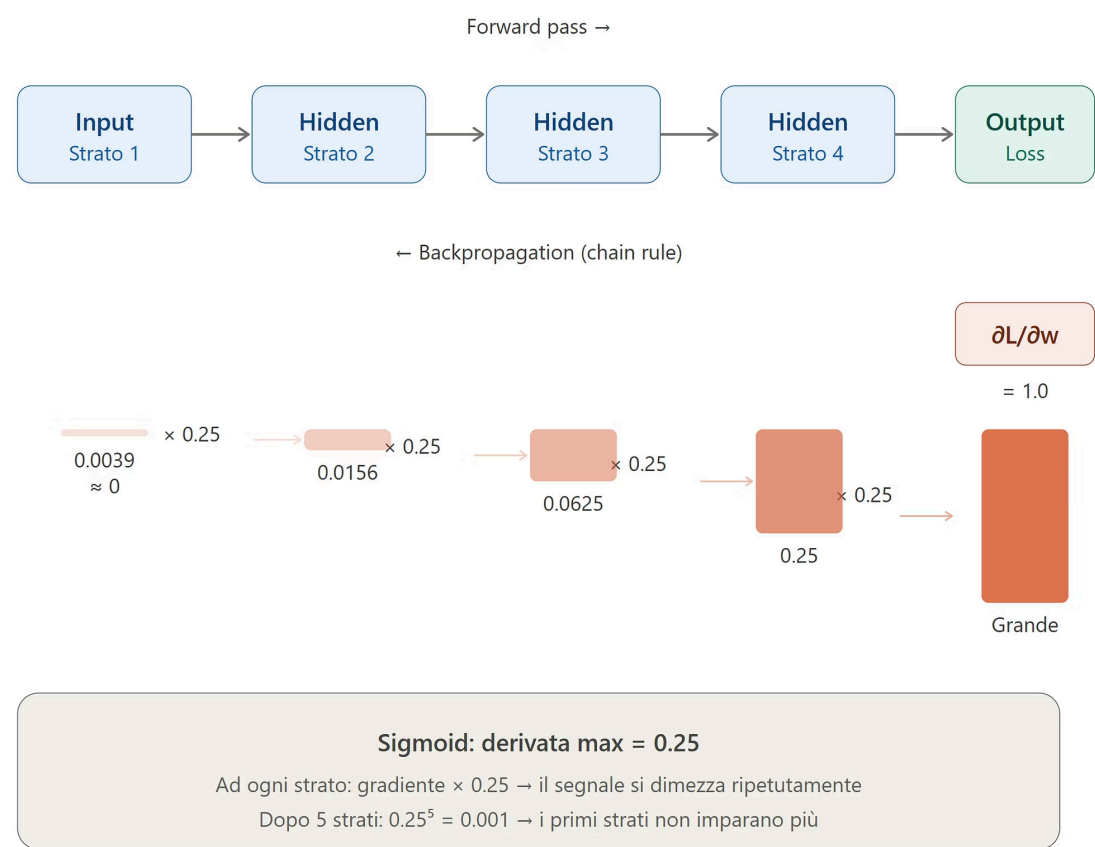
- **Problema con Modelli Non Lineari**

- Per la maggior parte dei modelli **non lineari**, inclusi le reti neurali **shallow** e **deep**, la funzione di loss è **non convessa**.
- Questo significa che possono esserci **minimi locali**, rendendo l'ottimizzazione più complessa.

Il Problema del Vanishing Gradient

Il concetto chiave

Durante la backpropagation, i gradienti vengono moltiplicati strato dopo strato tramite la chain rule. Se ad ogni strato il gradiente è un numero minore di 1 (ad esempio 0.2), queste moltiplicazioni ripetute fanno sì che il gradiente diventi esponenzialmente piccolo man mano che ci si avvicina agli strati iniziali.

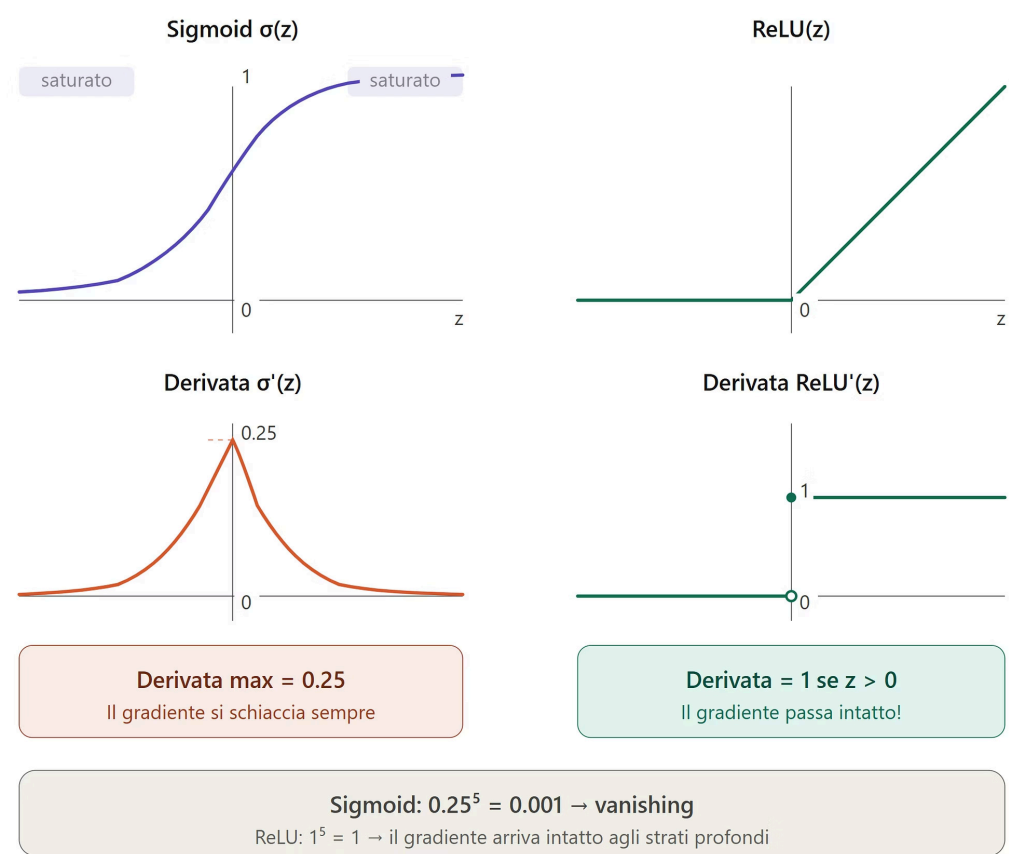


La conseguenza

Gli strati più vicini all'input smettono di imparare: i loro pesi non vengono aggiornati in modo significativo. La rete diventa "superficiale" nei fatti, anche se ha molti strati.

La causa e la soluzione: Sigmoid vs ReLU

La causa principale è la funzione di attivazione Sigmoid, la cui derivata ha un valore massimo di 0.25. La soluzione è usare ReLU, che ha derivata uguale a 1 per valori positivi.



Inizializzazione Intelligente dei Pesi

Il problema

Prima di iniziare il training, i pesi W di ogni hidden layer vengono inizializzati con valori casuali. Ma la scelta della scala di questi valori è cruciale:

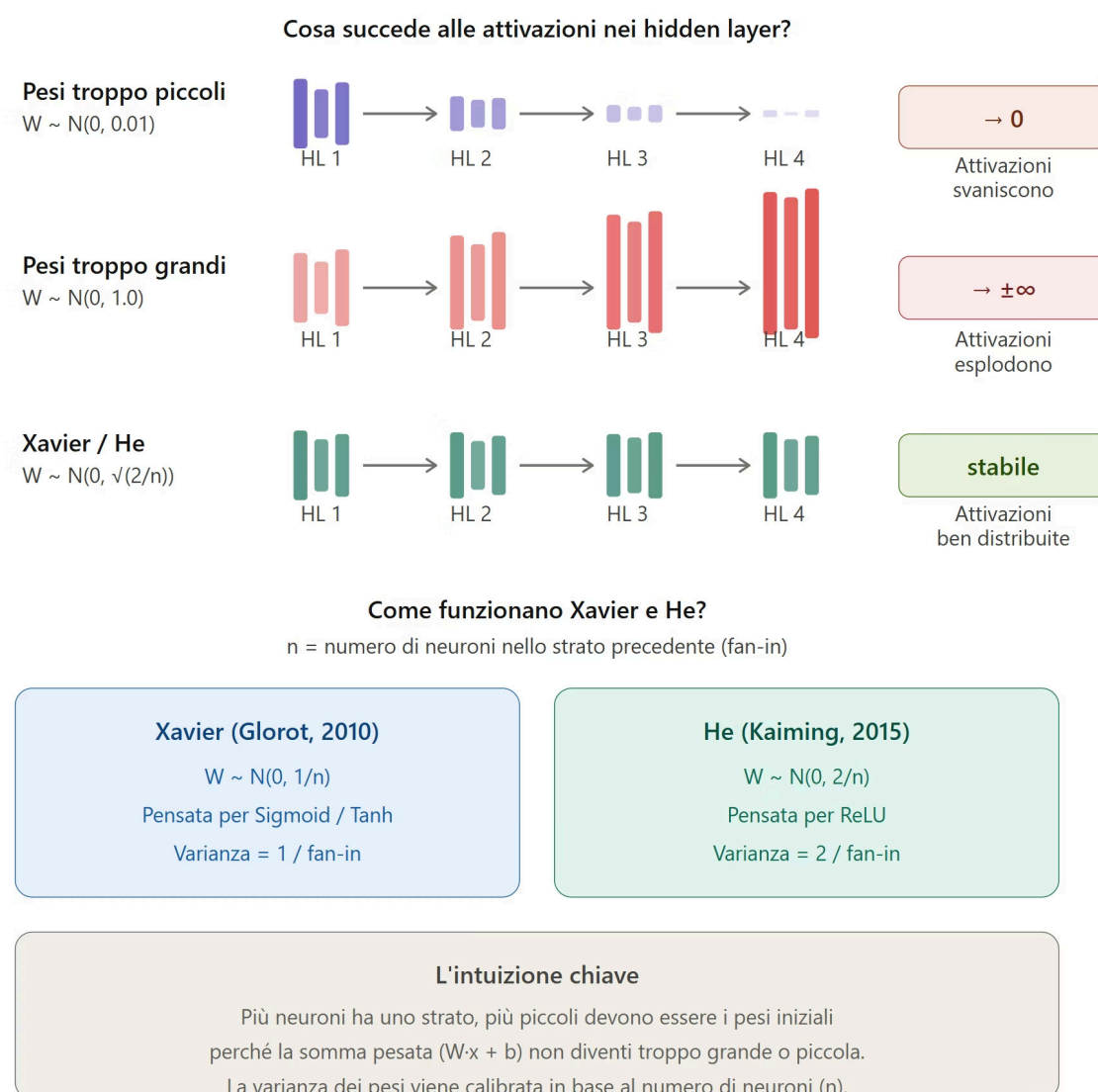
- **Pesi troppo piccoli** (es. $W \sim N(0, 0.01)$): le attivazioni si riducono ad ogni hidden layer, tendendo a zero. I gradienti svaniscono e la rete non impara.
- **Pesi troppo grandi** (es. $W \sim N(0, 1.0)$): le attivazioni crescono ad ogni hidden layer, saturando le funzioni di attivazione. I gradienti esplodono o diventano instabili.

L'intuizione

Ogni hidden layer calcola $h = \sigma(W \cdot x + b)$. La somma pesata $W \cdot x$ somma n termini (uno per ogni neurone dello strato precedente). Se i pesi sono troppo grandi, questa somma diventa enorme; se sono troppo piccoli, tende a zero.

L'idea chiave: più neuroni ha uno strato, più piccoli devono essere i pesi iniziali, per mantenere la varianza delle attivazioni stabile tra un hidden layer e il successivo.

Le due strategie



- **Xavier (Glorot, 2010)**: i pesi vengono estratti da una distribuzione $W \sim N(0, 1/n)$, dove n è il numero di neuroni nello strato precedente (fan-in). Pensata per funzioni di attivazione simmetriche come Sigmoid e Tanh.
- **He (Kaiming, 2015)**: i pesi vengono estratti da $W \sim N(0, 2/n)$. Il fattore 2 compensa il fatto che ReLU azzerava metà delle attivazioni (quelle negative), quindi serve il doppio della varianza per mantenere il segnale.

In pratica

Oggi la maggior parte dei framework (PyTorch, TensorFlow) applica automaticamente l'inizializzazione corretta in base alla funzione di attivazione scelta.

Batch Normalization

Il problema che risolve

Durante il training, le distribuzioni delle attivazioni di ogni strato cambiano continuamente man mano che i pesi vengono aggiornati. Questo fenomeno, chiamato *internal covariate shift*, costringe ogni strato a riadattarsi di continuo alle nuove distribuzioni in ingresso, rallentando l'apprendimento e amplificando il vanishing gradient.

L'intuizione

Batch Normalization è come "resettare" i valori delle attivazioni ad ogni strato, riportandoli ad una distribuzione stabile (media ≈ 0 , varianza ≈ 1) prima di passarli allo strato successivo. In questo modo i gradienti non incontrano valori estremi che li farebbero esplodere o svanire.

I 4 passi dell'algoritmo

1. Attivazioni sparse del mini-batch
2. Calcolo di media μ e varianza σ^2
3. Normalizzazione

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

4. Rescale con parametri apprendibili $y = \gamma \cdot \hat{x} + \beta$

Attivazioni stabili in output.

Perché γ e β ?

Se ci limitassimo a normalizzare, ogni strato produrrebbe sempre valori con media 0 e varianza 1, limitando l'espressività della rete. I parametri γ (scala) e β (traslazione) vengono appresi dal modello durante il training, permettendo alla rete di "decidere" la distribuzione ottimale per ogni strato.

Connessioni Residuali e Skip Connections

L'idea: aggiungere una scorciatoia

Si raggruppano uno o più hidden layer in un "blocco". La skip connection prende l'input x del blocco e lo somma direttamente all'output, saltando gli hidden layer intermedi.

- Senza skip: $y = F(x)$
- Con skip: $y = F(x) + x$

Dove $F(x)$ è la "trasformazione", cioè tutto ciò che gli hidden layer del blocco fanno ai dati.

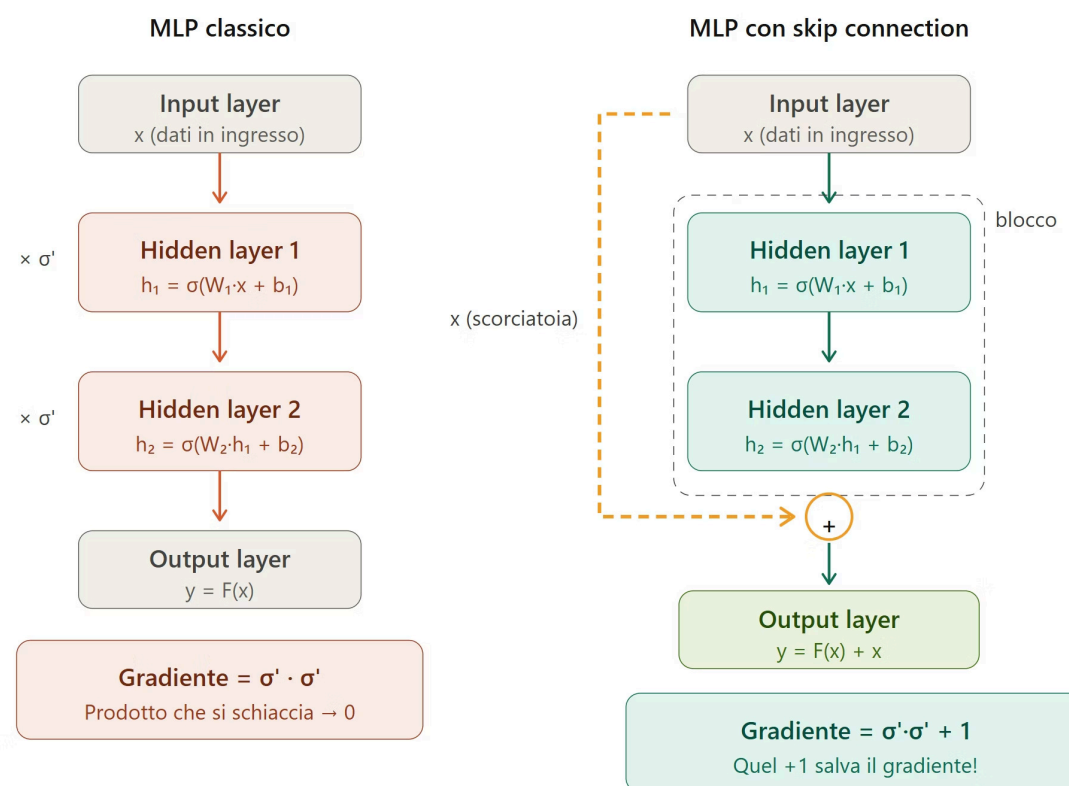
Un blocco può essere anche un solo hidden layer

Il caso più semplice di skip connection è un blocco con un singolo hidden layer:

- Hidden layer: $h = \sigma(W \cdot x + b)$
- Con skip: $h = \sigma(W \cdot x + b) + x$

In questo caso ogni hidden layer ha la propria scorciatoia. Nella pratica si usano più spesso blocchi da 2 hidden layer, per dare alla rete più "spazio" per imparare trasformazioni complesse dentro il blocco prima che il residuo venga sommato. Ma il principio matematico è sempre lo stesso: $y = F(x) + x$.

Perché funziona



Un vincolo tecnico

Per poter sommare x e $F(x)$, i due vettori devono avere la stessa dimensione. Questo significa che gli hidden layer del blocco devono avere lo stesso numero di neuroni dell'input. Se un hidden layer cambia il numero di neuroni, serve una piccola matrice di proiezione sulla skip connection: $F(x) + W_s \cdot x$.

Nota storica

Questo principio, introdotto con le reti ResNet nel 2015, è lo stesso che oggi troviamo nei Transformer alla base di GPT, BERT e tutti i moderni Large Language Model.

Dropout

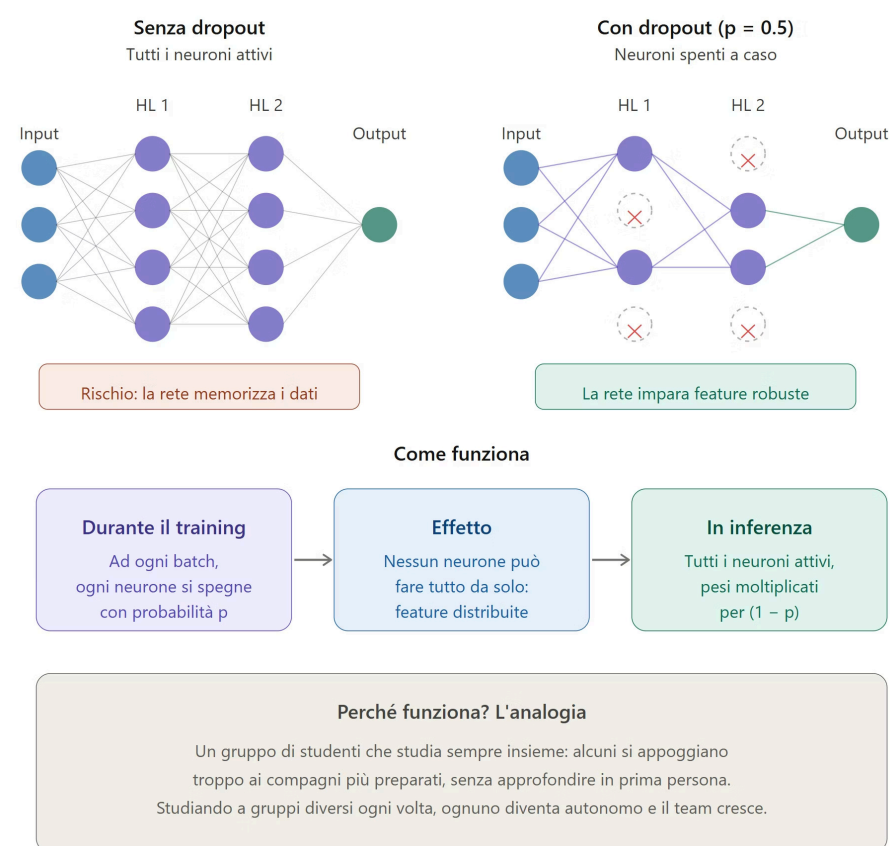
Il problema: overfitting

Un MLP con molti neuroni negli hidden layer ha abbastanza capacità per memorizzare i dati di training, invece di imparare pattern generali. Questo è l'overfitting: la rete funziona benissimo sui dati di training ma male su dati nuovi.

L'idea

Durante il training, ad ogni batch si "spengono" casualmente alcuni neuroni degli hidden layer, impostando la loro attivazione a zero. Ogni neurone ha una probabilità p (tipicamente 0.5) di essere disattivato. Ad ogni batch i neuroni spenti sono diversi, quindi la rete è costretta a non dipendere mai da un singolo neurone.

Come funziona



Training vs Inferenza

Questo è un dettaglio importante: il dropout si applica solo durante il training.

- **Training:** ad ogni batch, ogni neurone degli hidden layer viene spento con probabilità p . La rete si allena ogni volta con una "sotto-rete" diversa.
- **Inferenza (predizione):** tutti i neuroni sono attivi, ma i pesi vengono moltiplicati per $(1 - p)$ per compensare il fatto che durante il training c'erano meno neuroni attivi. Questo garantisce che la scala delle attivazioni rimanga coerente.

In pratica

- Il valore tipico è $p = 0.5$ per gli hidden layer (si spegne metà dei neuroni)
- Non si applica mai all'input layer né all'output layer
- In PyTorch: `nn.Dropout(p=0.5)`, da inserire tra gli hidden layer
- Dropout ha anche un leggero effetto di riduzione del vanishing gradient, perché semplifica i percorsi di backpropagation ad ogni batch.

Funzioni di Loss per la Classificazione

Il problema: dai logits alle probabilità

Una rete neurale produce in output numeri reali arbitrari, detti **logits**. Questi non sono probabilità: possono essere negativi, maggiori di 1, o qualsiasi valore.

Per interpretarli come probabilità servono due ingredienti:

- Una **funzione di trasformazione** (sigmoid o softmax)
- Una **loss function** che misuri quanto le probabilità predette sono lontane dalla realtà.

Binary Cross Entropy (BCE)

Quando si usa

Quando il problema ha **due sole classi** che si escludono a vicenda: spam/non spam, positivo/negativo.

Come funziona

Il modello produce un singolo logit z . La **sigmoid** lo trasforma in una probabilità $p \in (0, 1)$:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Se z è molto positivo: $p \approx 1$ (classe positiva)
- Se z è molto negativo: $p \approx 0$ (classe negativa)
- Se $z = 0$: $p = 0.5$ (massima incertezza)

La formula della loss

$$\text{BCE} = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

Dove $y \in \{0, 1\}$ è la classe vera e $p = \sigma(z)$ è la probabilità predetta.

Perché funziona

La formula ha due rami, ma solo uno è attivo alla volta:

- Se $y = 1$: la loss diventa $-\log(p)$. Più p è vicino a 1, più la loss è bassa.
- Se $y = 0$: la loss diventa $-\log(1 - p)$. Più p è vicino a 0, più la loss è bassa.

Il logaritmo **penalizza severamente le previsioni sicure ma sbagliate**: se il modello dice $p = 0.99$ ma la risposta vera è 0, la loss esplode.

Esempio numerico

Classe vera (y)	Predizione (p)	Loss
1	0.95	$-\log(0.95) = 0.05$
1	0.10	$-\log(0.10) = 2.30$
0	0.05	$-\log(0.95) = 0.05$
0	0.90	$-\log(0.10) = 2.30$

Categorical Cross Entropy (CCE)

Quando si usa

Quando le classi possibili sono **più di due** e **si escludono a vicenda**: cane/gatto/uccello, cifre 0-9, generi musicali.

Come funziona

Il modello produce **K logits** (uno per classe). La **softmax** li trasforma in K probabilità che sommano esattamente a 1:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

La formula della loss

$$\text{CCE} = -\sum_{i=1}^K y_i \cdot \log(p_i)$$

Dove y_i vale 1 solo per la classe corretta (codifica one-hot) e 0 per tutte le altre.

In pratica, dato che solo un y_i è diverso da zero, la formula si semplifica:

$$\text{CCE} = -\log(p_{\text{classe corretta}})$$

Messaggio chiave: la loss guarda solo la probabilità assegnata alla classe giusta. Se il modello dà 0.95 alla classe corretta, paga poco. Se dà 0.05, paga tantissimo.

Esempio numerico

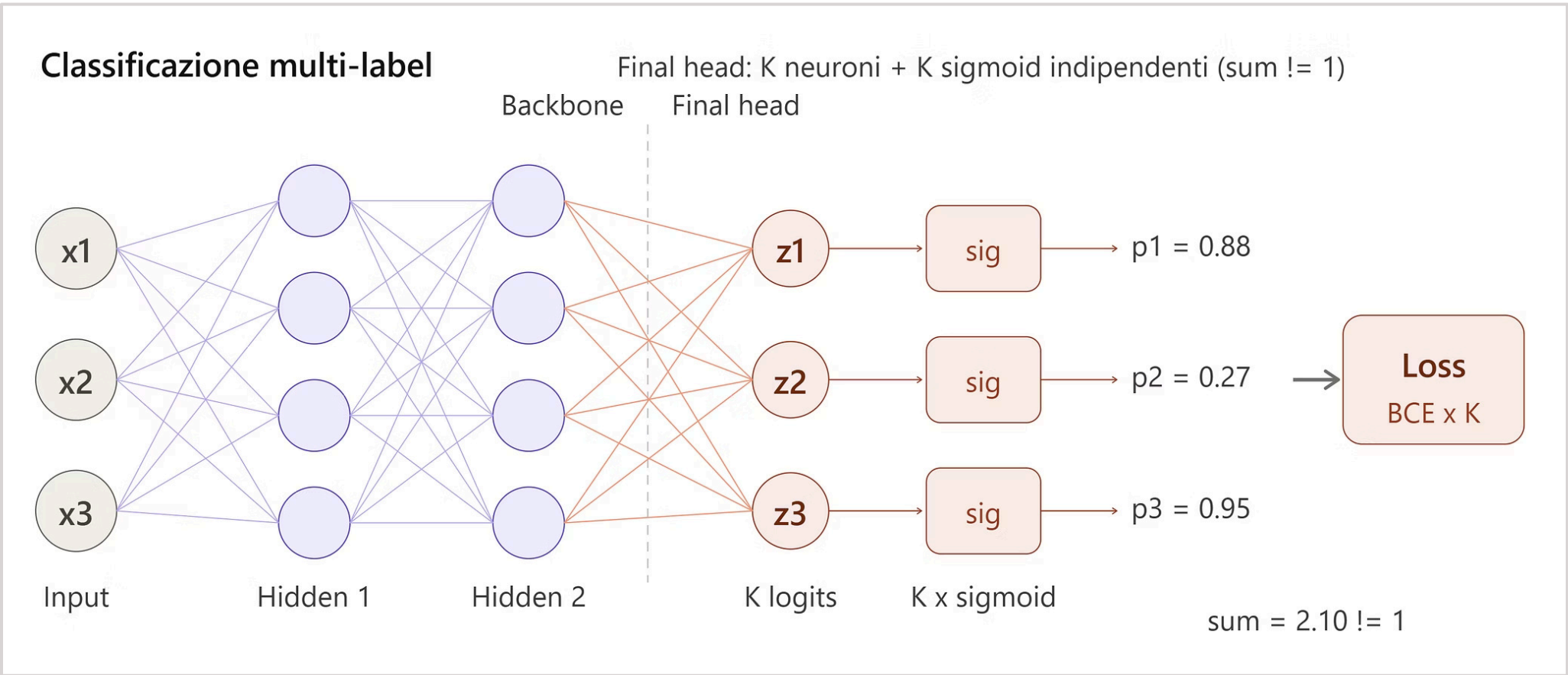
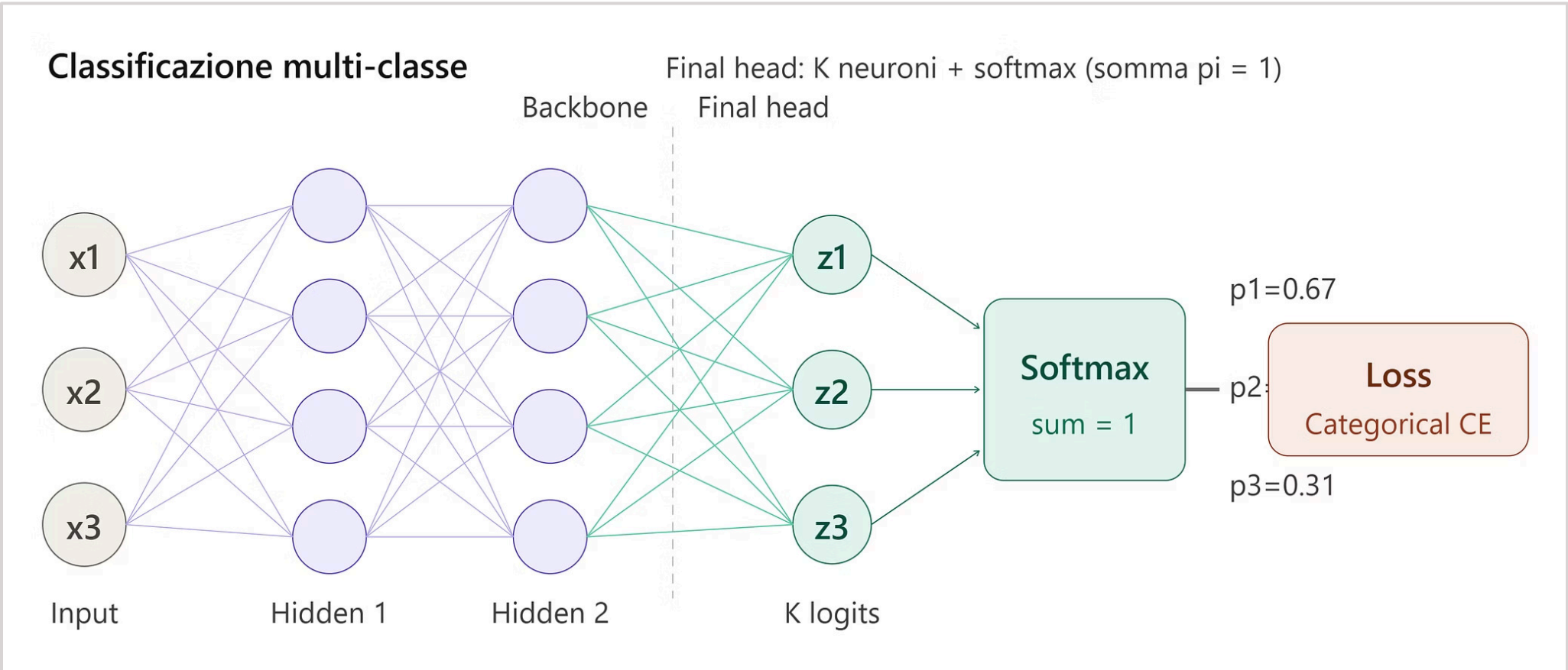
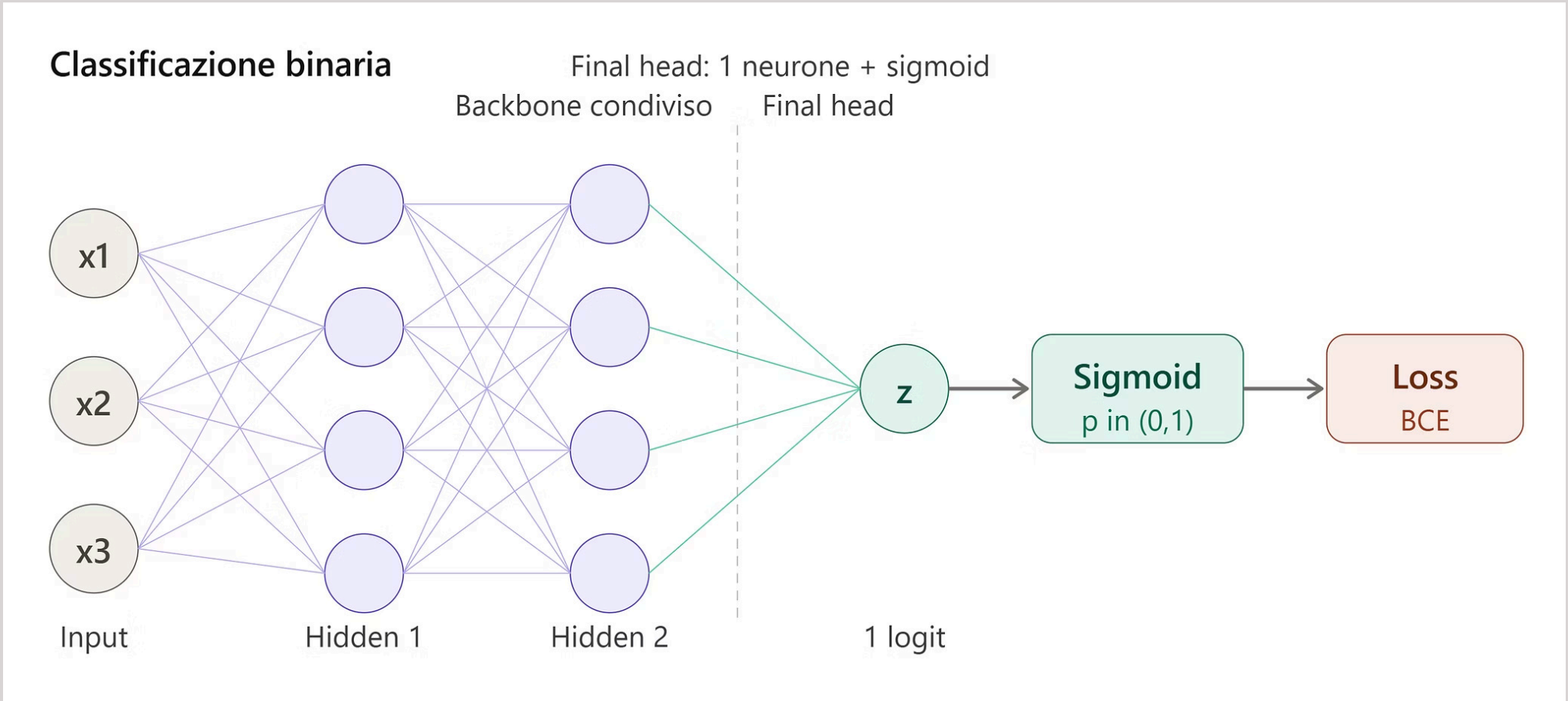
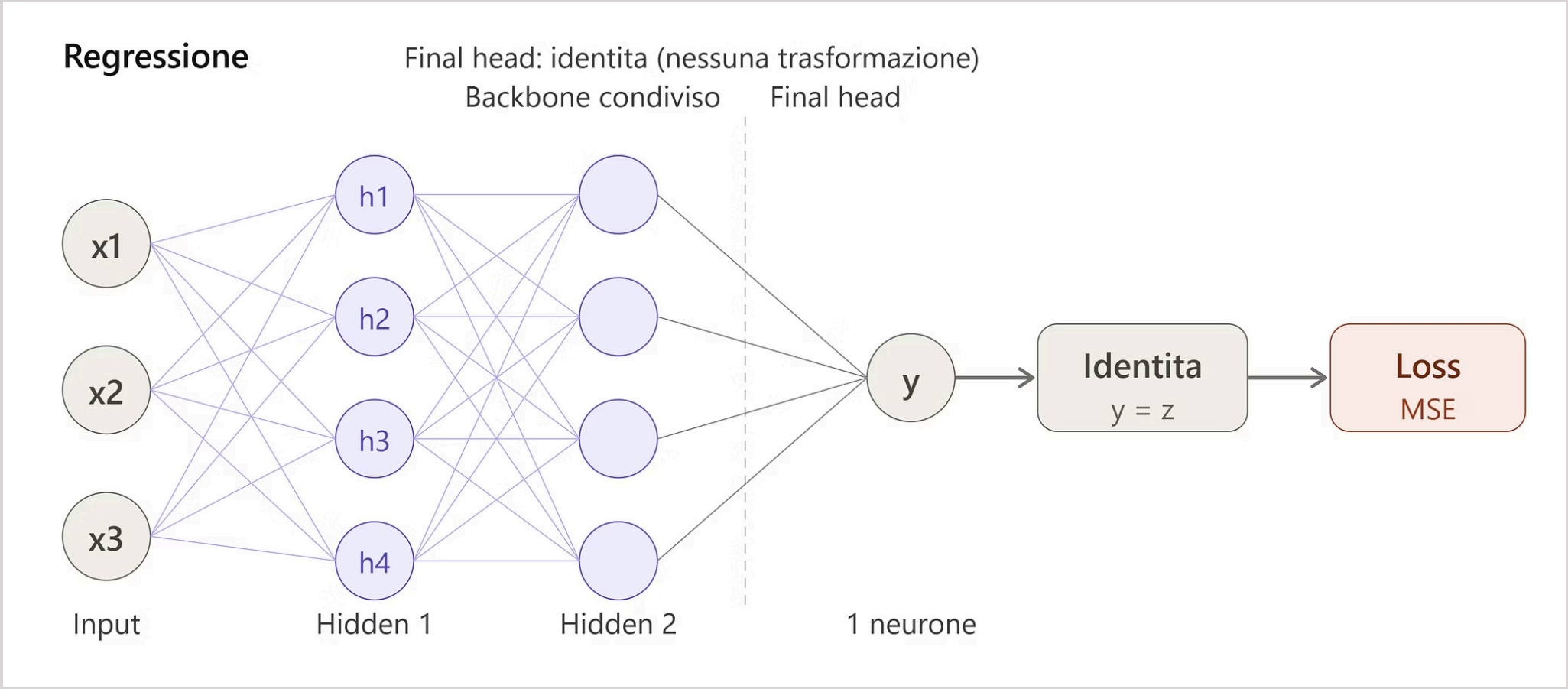
Un modello classifica un'immagine tra 3 classi. La classe vera è "cane":

	Cane	Gatto	Uccello
Logits (z)	3.0	-1.0	2.0
Softmax (p)	0.67	0.02	0.31
One-hot (y)	1	0	0

$$\text{CCE} = -\log(0.67) = 0.40$$

Solo p_{cane} conta nel calcolo della loss.

Adattare una Rete Neurale a Diversi Task



	Regressione	Binaria	Multi-classe	Multi-label
Neuroni output	1	1	K	K
Attivazione finale	Identità ($y = z$)	Sigmoid	Softmax	K x Sigmoid
Vincolo output	Nessuno	$p \in (0, 1)$	$\sum p_i = 1$	$\sum p_i \neq 1$
Loss function	MSE	Binary CE	Categorical CE	BCE per classe
Classi esclusive?	N/A	Sì	Sì	No
Esempio	Prezzo casa	Spam / non spam	Cane / gatto / uccello	Commedia + romantico

Temperatura

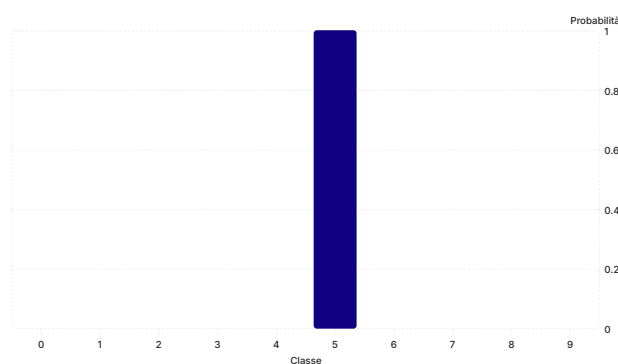
La funzione softmax standard può essere generalizzata introducendo un parametro T detto **temperatura**, che controlla quanto la distribuzione di probabilità risultante è "appuntita" o "piatta":

$$P(y = i) = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_j \exp\left(\frac{z_j}{T}\right)}$$

Per $T = 1$ si ottiene la softmax standard. Ma cosa succede quando variamo T ?

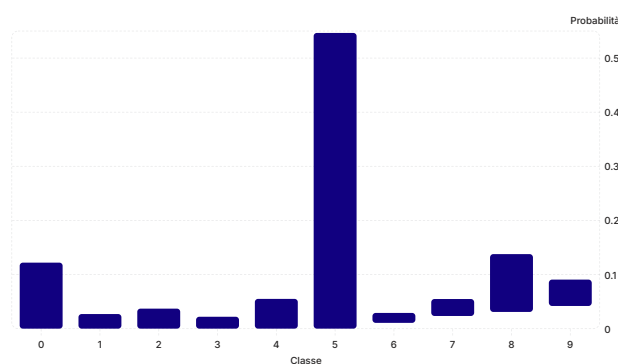
- $T \rightarrow 0$ (bassa temperatura): i logits vengono amplificati prima dell'esponenziale. La distribuzione diventa quasi deterministica: tutta la probabilità si concentra sulla classe con il logit più alto. La distribuzione ha **meno entropia**.
- $T = 1$: softmax standard, nessuna modifica ai logits.
- $T \rightarrow \infty$ (alta temperatura): i logits vengono schiacciati verso zero, le differenze tra classi scompaiono e la distribuzione tende verso l'uniforme — tutte le classi diventano equiprobabili. La distribuzione ha **più entropia**.

Bassa Temperatura ($T = 0.1$)



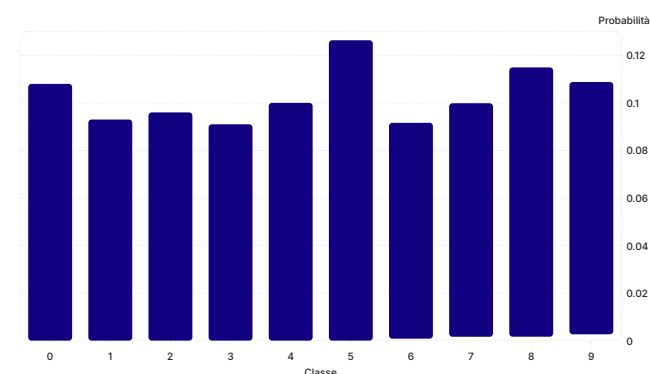
Distribuzione "appuntita": la probabilità si concentra sulla classe con il logit più alto. **Meno entropia.**

Standard ($T = 1$)



Softmax standard, con una chiara evidenziazione delle classi più probabili.

Alta Temperatura ($T = 10$)



Distribuzione "piatta": le classi diventano quasi equiprobabili. **Più entropia.**

Bassa Entropia



Entropia Moderata



Alta Entropia

- ❏ La temperatura è ampiamente utilizzata nei **modelli di linguaggio (LLM)** per controllare la creatività dell'output. Una temperatura bassa produce risposte più prevedibili e "sicure", concentrando la scelta sulle parole più probabili. Al contrario, una temperatura alta incoraggia il modello a esplorare opzioni meno probabili, generando risposte più creative e variegata.

Riferimenti

1. [Understanding Deep Learning](#), Simon J.D. Prince, 2024
2. ["The spelled-out intro to neural networks and backpropagation: building micrograd"](#), Andrej Karpathy, 2022
3. <https://nn-vis.noelith.dev/>
4. <https://karpathy.github.io/2026/02/12/microgpt/>
5. [MicroTorch](#)